Whole RTL Report

FPGA-Based MAC-Array Accelerator for Real-Time ECG Arrhythmia Classification

Phase 2 (Verilog RTL) and Vivado results

S Harish Adithya, Rajalakshmi Engineering College · 9 October 2026 · rev 1.0

1. Summary

This report documents the hardware half of the project: the Verilog design that classifies one ECG heartbeat as Normal or Abnormal using the trained, quantised network from the Golden Model report. It describes every module, how the modules are wired together, how the design was verified, and what Vivado made of it for the Zynq-7020 chip (xc7z020clg400-1). The complete source of the twelve synthesizable modules is printed in Appendix B.

Table 1. The hardware design at a glance.

Item	Result
Function	260 samples of one heartbeat go in (Q4.12), one decision comes out (Normal / Abnormal) together with both class scores (Q6.10).
Architecture	One shared multiply-accumulate unit runs all four layers (conv1, conv2, fc1, fc2) one after the other. A small layer table supplies each layer's shape, weight and bias start addresses, shifts and ReLU setting.
Source	12 synthesizable modules, 1,080 lines including comments (ecg_top 442 lines). Four layer-stage files and 16 testbench files are verification scaffolding only.
Speed	129,552 clocks per beat = 2.591 ms at 50 MHz, about 386 beats per second. The real-time need (200 beats per minute) is 3.3 beats per second, so the design is about 116 times faster than required.
Size (placed and routed)	394 LUTs (0.74 %), 213 flip-flops (0.20 %), 17.5 block-RAM tiles (12.5 %), 2 DSP blocks (0.91 %), 128 slices (0.96 %).
Timing (50 MHz)	Met. Setup slack +5.424 ns, hold slack +0.108 ns, no failing endpoints. Slowest path 14.049 ns of the 20 ns period (about 69 MHz possible).
Exactness	Bit-exact against the Python integer model: every value of every layer (4,066 per beat) and the final scores match, on 16 chosen beats and on three further sets of 16 random test beats.
Fault injection	111 deliberately broken copies of ecg_top.v: 100 caught, 11 not caught (all examined, none can change a result). 24 of 24 broken copies of layer_seq.v caught.
Gate level	The Yosys-synthesized netlist reproduces the answer key on 3 beats (normal, saturating, exact tie); a deliberately corrupted netlist is caught.

> **What is not covered yet.** No bitstream has been made and nothing has run on a board. Vivado timing and power were obtained with the design on its own (out-of-context, ideal clock). The Vivado-routed netlist has not been simulated. Section 9 lists every open item.

2. Architecture

2.1 How the design is used

From the outside, ecg_top is a very small machine. A controller (later the PYNQ wrapper) does three things:

- **Load the beat.** For each of the 260 samples, set `in_we` = 1, `in_waddr` = sample number (0 to 259) and `in_wdata` = the value (signed Q4.12), one sample per clock. Do not write while a beat is running.
- **Start.** Pulse `start_beat` for one clock while `beat_busy` is 0.
- **Wait for** `beat_done`**.** It is a one-clock pulse. From that clock on `score0` (Normal), `score1` (Abnormal) and `abnormal` are final and stay unchanged until the next beat reaches its last layer.

Table 2. Ports of ecg_top.

Port	Dir	Width	Meaning
clk	in	1	Clock (50 MHz target).
rst_n	in	1	Reset, active low, asynchronous. Returns every state machine and counter to its start. Stops a beat in progress.
in_we	in	1	Write enable of the input buffer.
in_waddr	in	9	Sample number, 0 to 259.
in_wdata	in	16	Sample value, signed Q4.12.
start_beat	in	1	One-clock pulse: run the four layers once. Ignored while a beat runs, except in the beat_done clock (back-to-back beats).
beat_busy	out	1	1 from the clock after an accepted start_beat until the clock before beat_done.
beat_done	out	1	One-clock pulse: results are final.
score0, score1	out	16 each	Class scores, signed Q6.10 (score0 = Normal, score1 = Abnormal). Registers, so they are stable between beats.
abnormal	out	1	1 when score1 > score0; 0 when score0 is higher and also on an exact tie.

2.2 The one idea behind the design

The network needs 121,408 multiplications per beat. Instead of building thousands of multipliers, the design builds **one** multiply-accumulate unit (mac_unit) and keeps it busy. Every output value of every layer (an *answer*) is the sum of a few products: it needs *taps* products, one per clock, added into a running total. When the total is finished it receives its bias, is shifted back to 16 bits, passes through ReLU, and is stored. Then the next answer starts. All four layers work this way; only the number of taps, the number of answers, the weights and the places the data is read from and written to differ. That is why the whole design is a single set of counters, one ROM, a few small memories and one arithmetic unit, driven by a table of settings.

Table 3. Work in one beat. Clocks per answer = taps + 2; clocks per layer = answers × (taps + 2) + 2.

Layer	Answers per beat	Taps per answer	Multiplies	Clocks per answer	Clocks per layer
conv1	8 × 256 = 2,048	5	10,240	7	14,338
conv2	16 × 124 = 1,984	40	79,360	42	83,330
fc1	32	992	31,744	994	31,810
fc2	2	32	64	34	70
Total	4,066		121,408		129,548 (+4 = 129,552)

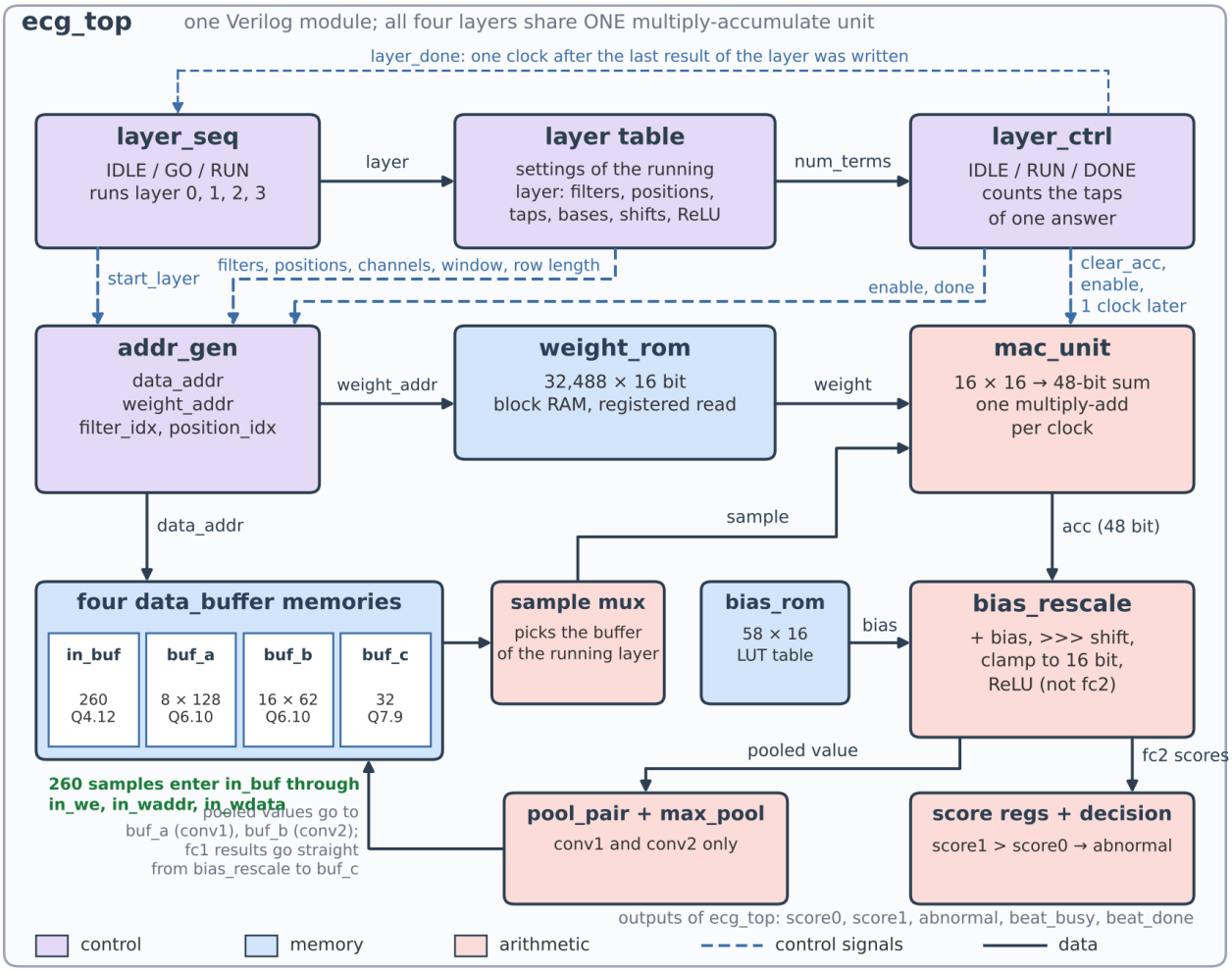


Figure 1. Block diagram of ecg_top. Purple blocks control the sequence, blue blocks store data, red blocks do arithmetic. Dashed lines are control signals, solid lines carry data and addresses.

2.3 The modules

Table 4. The twelve synthesizable modules (line counts include comments).

Module (file)	Lines	Kind	What it does
ecg_top	442	top	Wires everything, holds the layer table, the one-clock-delay registers, the write addresses, the sample mux and the score registers.
layer_seq	93	control	Runs layer 0, 1, 2, 3 one after another; gives start_layer, beat_busy, beat_done.
layer_ctrl	66	control	Counts the taps of one answer; gives clear_acc, enable, done.
addr_gen	137	control	Turns the tap count into data and weight addresses with running base addresses, no multipliers.
weight_rom	41	memory	All 32,488 weights in block RAM (registered read), loaded from weights.hex.
bias_rom	35	memory	58 biases as a LUT table (instant read), loaded from biases.hex.
data_buffer	55	memory	One small memory, used four times: input (260), A (1,024), B (992), C (32).
mac_unit	31	arithmetic	16 × 16 multiply-accumulate into a 48-bit register.

Module (file)	Lines	Kind	What it does
bias_rescale	63	arithmetic	Adds the bias, shifts right, clamps to 16 bits, applies ReLU (not for fc2). Combinational.
max_pool	30	arithmetic	Keeps the larger of two numbers. Combinational.
pool_pair	67	arithmetic	Holds the first number of a pair until its partner arrives, then calls max_pool.
decision	20	arithmetic	abnormal = (score1 > score0), signed compare. Combinational.

Four more files (conv1_stage.v, conv2_stage.v, fc1_stage.v, fc2_stage.v) wire the same modules together one layer at a time. They were used to prove each layer on its own before the layers were merged into ecg_top, and are described in Section 5.11. They are not part of the synthesized design.

3. The layer table and the memories

3.1 Settings of each layer

Everything that differs between the layers is in one place: a table inside ecg_top.v, selected by the layer number from layer_seq. The numbers were generated by `make_answer_key.py` from the quantised network and are the same ones the Python model uses.

Table 5. The layer table (ecg_top.v).

Setting	conv1	conv2	fc1	fc2	Meaning
num_filters	8	16	32	2	Filters (conv) or neurons (fc).
num_positions	256	124	1	1	Positions the window takes (conv); 1 for fc.
num_channels	1	8	1	1	Input rows (conv2 reads 8 pooled rows).
kernel	5	5	992	32	Window length; for fc it is the number of inputs.
row_len	0	128	0	0	Distance between input rows (only conv2 needs it).
num_terms (taps)	5	40	992	32	Products per answer = channels × kernel.
w_base	0	40	680	32,424	First weight of this layer in weight_rom.
b_base	0	8	24	56	First bias of this layer in bias_rom.
bias_shift	12	10	10	9	Left shift that aligns the bias with the accumulator.
out_shift	16	15	16	14	Right shift that brings the sum back to 16 bits.
ReLU	yes	yes	yes	no	fc2 delivers the raw class scores.
Output format	Q6.10	Q6.10	Q7.9	Q6.10	Format of the stored result.

The shifts follow from the number formats chosen in the quantisation work: the accumulator holds (input fractional bits + weight fractional bits) fractional bits, the bias is shifted left to that position, and the sum is shifted right to the layer's output format. For example, conv1 has 12 + 14 = 26 fractional bits in the accumulator, so the bias moves left by 12 (14 → 26) and the sum moves right by 16 (26 → 10, Q6.10).

3.2 Weights and biases

Table 6. Where each layer's parameters sit.

Layer	Weights	Count	weight_rom addresses	Biases	bias_rom addresses
conv1	8 × 1 × 5	40	0 – 39	8	0 – 7
conv2	16 × 8 × 5	640	40 – 679	16	8 – 23
fc1	32 × 992	31,744	680 – 32,423	32	24 – 55
fc2	2 × 32	64	32,424 – 32,487	2	56 – 57
Total		32,488		58	

Each file is one 4-digit hex number per line (16-bit two's complement), written by `export_rom_files.py` from the saved quantised weights. In sequence they are conv1 (Q2.14) followed by conv2, fc1 and fc2 (Q1.15). weight_rom holds 63.5 KiB of weights; this is the single largest cost of the design (Section 7).

3.3 The four data buffers

One module, data_buffer, is used four times with different depths. Each has one write port and one read port. The read is registered: the value appears one clock after the address, the same delay as weight_rom, so a sample and its weight arrive together.

Table 7. The data buffers.

Buffer	Words	Format	Holds	Written by	Write address	Read by
in_buf	260	Q4.12	the beat's samples	the outside (in_we, in_waddr)	in_waddr	conv1
buf_a	1,024	Q6.10	conv1 result, pooled: 8 × 128	conv1 (via pool_pair)	filter × 128 + position / 2	conv2
buf_b	992	Q6.10	conv2 result, pooled: 16 × 62	conv2 (via pool_pair)	filter × 62 + position / 2	fc1
buf_c	32	Q7.9	fc1 result	fc1	filter (neuron number)	fc2

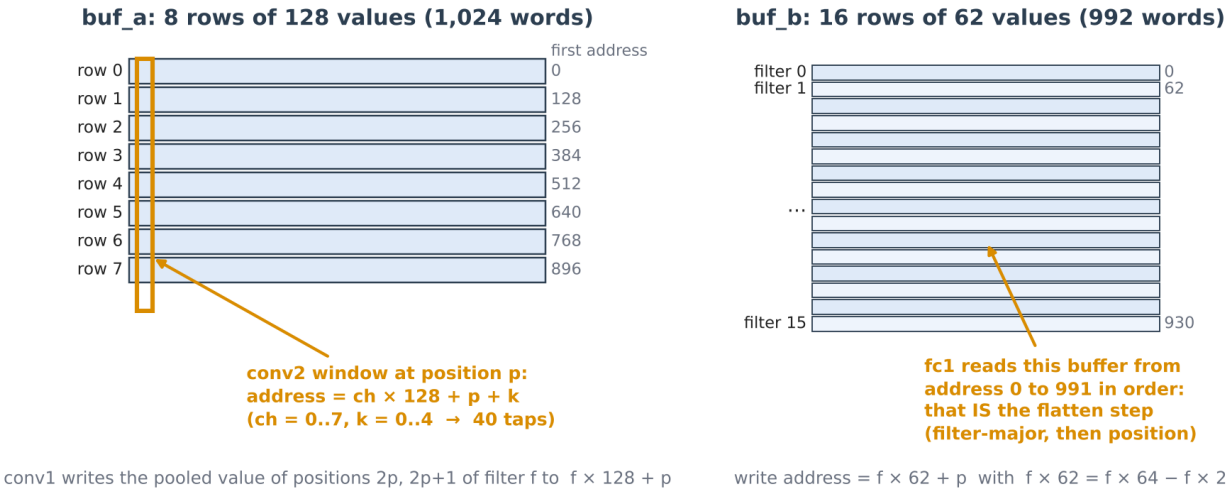


Figure 2. Left: buf_a holds the pooled conv1 result as 8 rows of 128. conv2 reads the same 5-wide window across all 8 rows; the address is row × 128 + position + k. Right: buf_b holds the pooled conv2 result as 16 rows of 62. Because it is written in this order, fc1 can read it from address 0 to 991 directly: the flatten step costs nothing.

conv2's pooled-position address uses filter × 62. The design builds this as filter × 64 − filter × 2, using only wiring and one subtractor, so no multiplier is needed anywhere in the address logic. The two score registers (score0, score1, Q6.10) take fc2's two results; they are registers rather than a buffer.

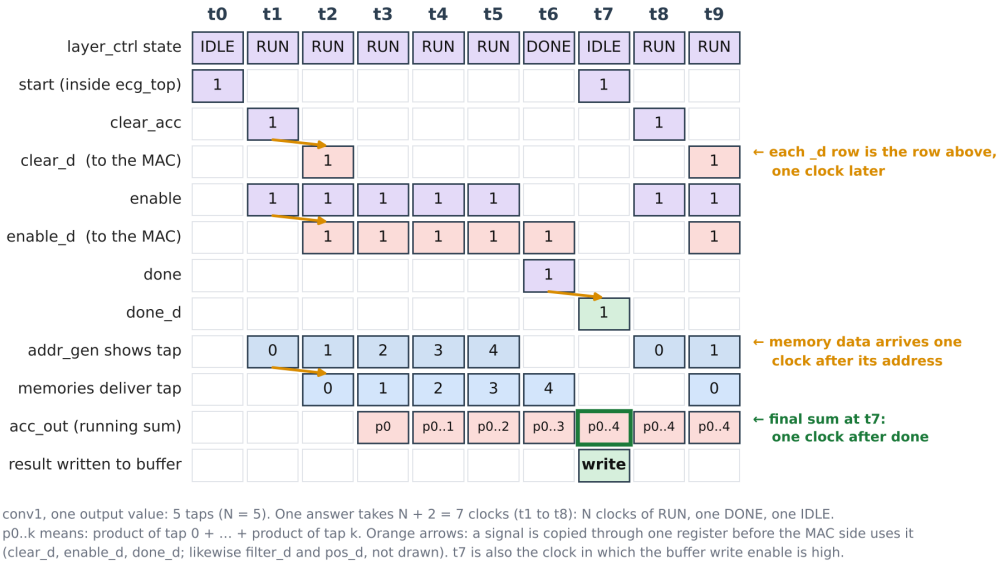
4. Timing: the one-clock-delay rule

Block RAM reads are synchronous: the address is presented in one clock and the data appears in the next. Both weight_rom and every data_buffer work like this. As a result, when addr_gen shows the address of tap *n*, the sample and the weight of tap *n* are only available one clock later. Everything on the arithmetic side of the design must therefore be one clock late, too. ecg_top does this with a handful of delay registers:

Table 8. The delay registers in ecg_top.

Signal on the control side	Delayed copy used on the arithmetic side	Used for
clear_acc	clear_d	Tells mac_unit to start a fresh sum.
enable	enable_d	Tells mac_unit to add this clock's product.
done	done_d	Marks the clock in which the finished sum is in the accumulator; enables every result write.
last_output	last_d	Marks the final answer of a layer (to produce layer_done).
filter_idx	filter_d	Bias address, write addresses, which score register.
position_idx	pos_d	Write address and the even/odd flag for pooling.

layer_ctrl and addr_gen are **not** delayed: they create the addresses, so they run one clock ahead of the arithmetic. mac_unit registers its sum, so when done_d is 1 the accumulator holds the finished total, and filter_d and pos_d still describe that same output (addr_gen moves on at the end of the done clock). Figure 3 shows one conv1 answer, clock by clock.



conv1, one output value: 5 taps (N = 5). One answer takes N + 2 = 7 clocks (t1 to t8): N clocks of RUN, one DONE, one IDLE.
p0..k means: product of tap 0 + … + product of tap k. Orange arrows: a signal is copied through one register before the MAC side uses it (clear_d, enable_d, done_d; likewise filter_d and pos_d, not drawn). t7 is also the clock in which the buffer write enable is high.

Figure 3. One answer of conv1 (5 taps) in the clocks t0 to t9, with the delayed signals directly below the signals they follow. The final sum is in acc_out at t7, the same clock in which done_d is 1 and the result is written.

4.1 Where the clock counts come from

- **One answer = taps + 2 clocks.** layer_ctrl spends one clock in RUN per tap, one in DONE and one in IDLE before the next answer starts.
- **One layer = answers × (taps + 2) + 2 clocks**, measured from the start_layer clock to the layer_done clock. The extra 2 are the done_d register and the register that creates layer_done, so a layer is only reported finished one clock after its last result was written.
- **Between layers: 1 clock** (layer_done to the next start_layer), and **1 clock** from start_beat to the first start_layer.
- **Whole beat:** 14,338 + 83,330 + 31,810 + 70 = 129,548, plus 4 (one start-up clock and three hand-overs) = **129,552**.

Table 9. Predicted and tested clock numbers, counted from the start_beat clock (= 0).

Layer	start_layer at clock	layer_done at clock
conv1	1	14,339
conv2	14,340	97,670
fc1	97,671	129,481
fc2	129,482	129,552 (= beat_done)

The testbench checks every one of these numbers on every run, so a one-clock error anywhere in the design is caught (two of the fault-injection variants in Section 6.4 do exactly that).

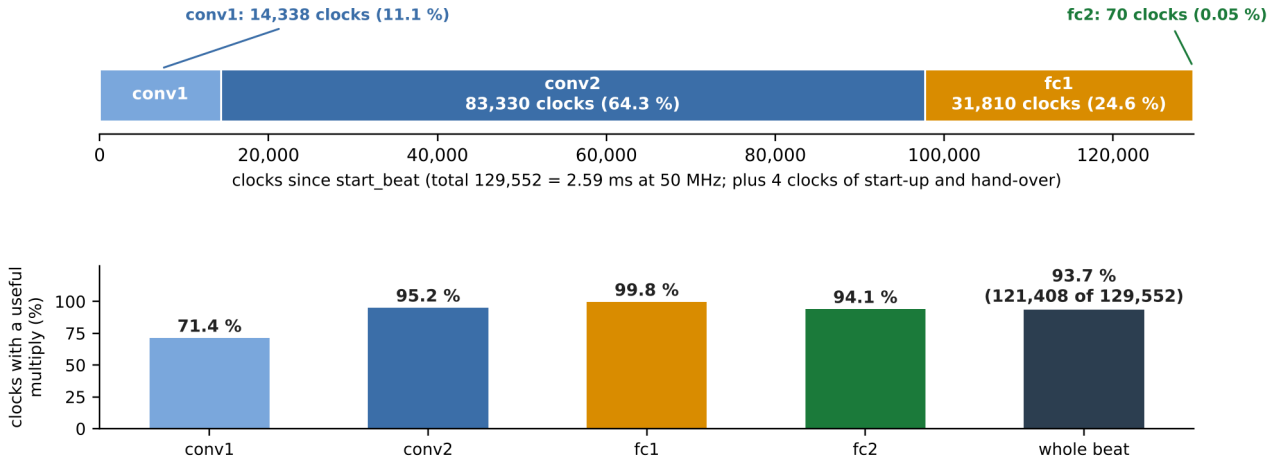


Figure 4. Top: where the 129,552 clocks of one beat go. Bottom: share of clocks in which the multiplier does a useful product (taps / (taps + 2)). Overall, 93.7 % of all clocks do a real multiply; the rest is bookkeeping between answers.

5. Module reference

Each module is described the same way: what it is for, its ports, how it works, and how it was verified. Verification numbers are from the simulation runs made for this report (Icarus Verilog) on the files in the project folder; Section 6 puts them together. Where a Vivado number is known for a module it is given as well.

5.1 layer_seq: which layer runs

layer_seq (93 lines) knows nothing about arithmetic. It only says which layer is running (0 = conv1, 1 = conv2, 2 = fc1, 3 = fc2), gives one start pulse when a layer should begin, and waits for the datapath to say that the layer is finished. Then it moves on.

Port	Dir	Width	Meaning
clk, rst_n	in	1 + 1	Clock; reset (active low, asynchronous): back to IDLE, layer 0.
start_beat	in	1	Run the four layers once.
layer_done	in	1	From the datapath: the running layer is finished.
layer	out	2	Number of the running layer.
start_layer	out	1	One-clock pulse: the datapath starts layer 'layer' now.
beat_busy	out	1	A beat is running.
beat_done	out	1	One-clock pulse in the same clock as the fourth layer_done.

- Three states: IDLE (waiting), GO (lasts exactly one clock; start_layer is high), RUN (waiting for layer_done).
- start_layer comes exactly one clock after start_beat (layer 0) and exactly one clock after each layer_done (layers 1 to 3). The layer number changes in the clock *before* start_layer, so the layer table is stable when the layer starts.
- layer_done is listened to only in RUN; a stray layer_done in IDLE or GO is ignored.
- start_beat is accepted only when idle or in the very clock in which the beat finishes, so beats can follow each other without a gap. At any other time it is ignored.
- Between beats the layer number stays at 3; the next accepted start_beat sets it back to 0.

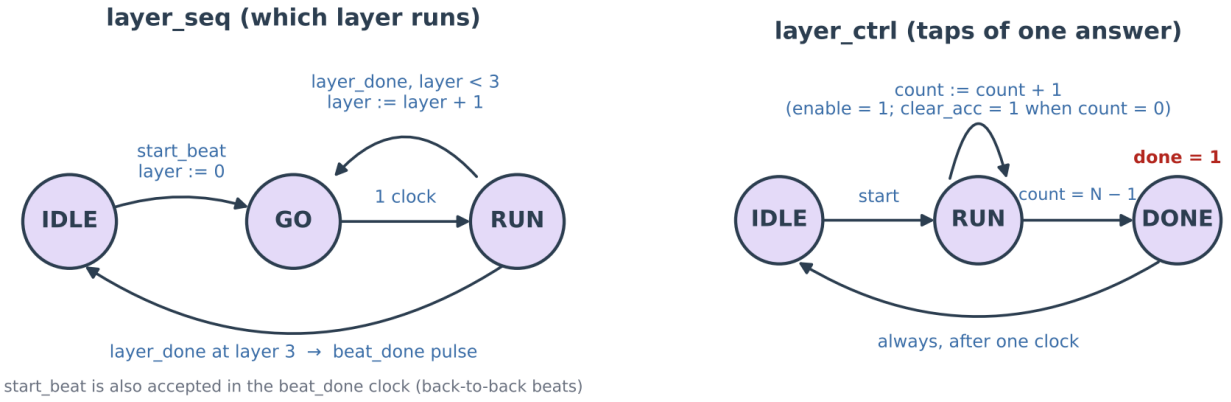


Figure 5. The two state machines. Left: layer_seq steps through the four layers. Right: layer_ctrl counts the taps of one answer.

Verification. A random test of 400,000 clocks compared all four outputs against an independent model in every clock (1,600,000 comparisons). It produced 11,118 complete beats, each with exactly four start_layer pulses; 44,961 start_layer pulses in all; 70,502 start_beat pulses while busy (all ignored); 1,427 stray layer_done pulses while idle and 5,688 in GO (all ignored); 5,860 beats started from idle and 5,458 started in the beat_done clock; and 209 resets, 203 of them in the middle of a beat. Zero errors. A separate campaign of 24 deliberately broken copies was caught in full (Section 6.4).

5.2 layer_ctrl: counting the taps of one answer

layer_ctrl (66 lines) steps the multiply-accumulate unit through the N taps of *one* output value (one filter position, or one neuron). It has three states, IDLE, RUN and DONE (Figure 5, right), and its outputs depend only on the state and the counter. The counter has 11 bits (parameter CNT_W), enough for fc1's 992 taps; num_terms must be at least 1.

Port	Dir	Width	Meaning
start	in	1	Begin an answer (accepted in IDLE).
num_terms	in	11	Taps in this answer: 5, 40, 992 or 32.
clear_acc	out	1	1 on the first tap only: start a fresh sum.
enable	out	1	1 on every tap (state RUN).
busy	out	1	1 in RUN. ecg_top uses it, with done, to see when layer_ctrl is idle.
done	out	1	1 for one clock after the last tap (state DONE).
term_idx	out	11	The counter. Not used by ecg_top (addr_gen keeps its own counters).

Verification. The testbench connects layer_ctrl to the real mac_unit and two small memories and compares the finished accumulator with an independently computed dot product for N = 1, 5, 40, 32 and 992 (the four real layer sizes and a single tap), then for a re-use with new data, and for the worst case: all 992 products equal to $(-32768)^2 = 2^{30}$, which gives $992 \times 2^{30} = 1{,}065{,}151{,}889{,}408$. That value needs 41 bits, so the 48-bit accumulator cannot overflow. Monitors also check that every run has exactly one clear_acc, N enables and one done pulse. Three deliberate faults were caught: an off-by-one in the last-tap test, clear_acc on every clock, and clear_acc never.

5.3 addr_gen: where to read, every clock

addr_gen (137 lines) is shared by all four layers. In every clock in which the MAC does a multiply it says which data word and which weight to read. The shape of the layer comes in as inputs (the layer table), so nothing in the module is specific to one layer. It counts four nested loops, innermost first:

- **k** (0 to kernel − 1): the step inside one row of the window;
- **ch** (0 to channels − 1): which input row; together, k and ch make one answer's *tap* = ch × kernel + k;
- **pos** (0 to positions − 1): where the window sits;
- **filt** (0 to filters − 1): which filter or neuron.

```
data_addr   = ch * row_len + pos + k
weight_addr = filt * (channels * kernel) + tap
```

There are no multipliers. Instead of computing ch × row_len and filt × taps each time, the module keeps running totals: cbase (= ch × row_len, increased by row_len when the window moves to the next row), waddr (the weight to read now, simply increased by 1 per tap) and wbase (the first weight of the current filter). At the end of an answer waddr jumps back to wbase (the next position uses the same weights again) or, after the last position, continues into the next filter's weights, which follow directly in the ROM. A fully connected layer uses the same module with positions = 1 and channels = 1, so kernel is the number of inputs and data_addr is simply the tap number.

Table 10. Counter widths (parameters of addr_gen).

Parameter	F_W	P_W	C_W	K_W	DA_W	WA_W
Width (bits)	6	9	4	10	11	16
Holds up to	63	511	15	1,023	2,047	65,535
Largest value needed	32	256	8	992	1,024	31,744

Port	Dir	Width	Meaning
init	in	1	Synchronous: go back to the very first tap (ecg_top connects start_layer).
step	in	1	A tap is consumed this clock (layer_ctrl enable, undelayed).
next_output	in	1	One answer has just finished (layer_ctrl done, undelayed). Never high together with step.
num_filters ... row_len	in	6, 9, 4, 10, 11	The layer's shape, from the layer table.
data_addr	out	11	Address of the data word of the tap used this clock.
weight_addr	out	16	Address of the weight, relative to the layer's own first weight.
filter_idx, position_idx	out	6, 9	Stay on the answer just finished until the end of the done clock, so the result can be stored and its bias looked up.
last_output	out	1	High while the final answer of the layer is in progress.

Size. Standalone (open-source Yosys): 72 flip-flops, 122 LUTs, 28 carry cells, no DSP, no block RAM. In Vivado's hierarchy view: 81 LUTs, 62 flip-flops, 35 slices (Vivado removes counter bits that can never be reached).

Verification. Six scenarios, each comparing every (data address, weight address) pair with an independently computed list and checking the totals against Python: conv1 (10,240 multiply clocks), conv1 again without init, conv1 after an init in the middle of the layer, conv2 (79,360), fc1 (31,744) and fc2 (64); after each layer the counters must have wrapped back to the start. All pass. Eight deliberate faults were caught: channel base never advances, weights not re-read for the next position, off-by-one on the last position, data address ignores the position, init does not reset the position, last_output asserted too early, next filter starts one weight late, and filter counter never wraps.

5.4 weight_rom and bias_rom

weight_rom (41 lines) stores all 32,488 weights, conv1 first, then conv2, fc1, fc2 (start addresses 0, 40, 680, 32,424), loaded from weights.hex with `$readmemh`. It carries the attribute `rom_style = "block"` and has a registered read: the weight shows up one clock after its address. The address is the layer's w_base plus addr_gen's weight_addr (largest value 32,424 + 63 = 32,487; addresses 32,488 and above do not exist).

bias_rom (35 lines) stores the 58 biases (start addresses 0, 8, 24, 56) loaded from biases.hex. Unlike the weight ROM its read is instant (combinational): 58 numbers are small enough for the tools to build from ordinary LUTs, so no delay has to be compensated. Its address is b_base + filter_d, which is exactly what is on the bus in the done_d clock, when bias_rescale needs the bias.

Verification. weight_rom: every one of the 32,488 addresses read back; four per-layer checksums (each weight multiplied by its position, so a swapped, shifted, dropped or changed weight cannot hide), the sum and the XOR of all weights, and 28 spot values, all computed by Python straight from the saved weight file and not from the hex file; 40,000 random addresses with a new address every

clock; the first and last address alternately; and the one-clock delay itself (data must not change before the clock edge). bias_rom: all 58 biases against Python, 5,000 random addresses, and the sum of all biases (35,549) equal to Python's.

5.5 data_buffer: the four activation memories

data_buffer (55 lines) is one simple memory with one write port and one read port on the same clock, parameterised by DEPTH (the address width is the base-2 logarithm of DEPTH, rounded up). When we = 1, wdata is stored at waddr at the next clock edge. The read is registered: the value appears one clock after raddr. If the same address is written and read in the same clock, the read returns the *old* value (read-first). The design never does this on purpose, because a layer reads one buffer while it writes a different one, but the behaviour is fixed so it is never a surprise. There is no reset and no initial content: a word that was never written is undefined, so a layer never reads a word before the layer before it has written it. The tools decide whether an instance becomes block RAM or distributed (LUT) RAM; because the read is registered in the code, both behave identically clock for clock.

Table 11. data_buffer verification: random writes and reads against a model, at the four depths used.

Instance	DEPTH	Random checks	Same-address read+write events	Result
buf_c	32	22,694	5,046	pass
buf_b	992	100,454	4,826	pass
buf_a	1,024	149,638	4,877	pass
in_buf	260	196,682	4,796	pass

Total 469,468 checks, 0 errors.

5.6 mac_unit: the multiplier

mac_unit (31 lines) multiplies a signed 16-bit sample by a signed 16-bit weight (a 32-bit product, sign-extended to 48 bits) and adds it to a 48-bit register. Parameters: DATA_WIDTH = 16, WEIGHT_WIDTH = 16, ACC_WIDTH = 48. The module is format-agnostic; it never sees where the binary point is.

Table 12. Behaviour of mac_unit.

rst_n	clear_acc	enable	Next acc_out	Meaning
0	x	x	0	Reset.
1	1	1	data_in × weight_in	First tap: start a fresh sum with this product.
1	1	0	0	Clear without adding.
1	0	1	acc_out + data_in × weight_in	Accumulate.
1	0	0	acc_out	Hold (no update).

Why 48 bits. The largest possible product is $(-32768)^2 = 2^{30}$. Summing 992 of them (fc1) gives 1,065,151,889,408, which needs 41 bits. The 48-bit register therefore cannot overflow, whatever the data. Overflow is dealt with once per answer, by the saturation in bias_rescale.

Vivado. Vivado used 2 DSP48E1 blocks for the multiplier and built the 48-bit accumulate in ordinary logic (48 flip-flops, 80 LUTs). The earlier report had expected one DSP block to hold both multiply and accumulate, and the open-source Yosys run did use one. This is a mapping choice of the tool, with no functional effect; it costs one more of the 220 DSP blocks.

Verification. The unit test feeds three known values (0.25, −0.125, −0.25) and checks the sum, −0.125 = −134,217,728 as a raw integer, including sign extension. layer_ctrl_tb exercises the unit with random data and the worst case (Section 5.2), and every whole-chip test checks it indirectly: every finished sum of every layer feeds the compared values.

5.7 bias_rescale: bias, shift, clamp, ReLU

bias_rescale (63 lines) turns the finished 48-bit sum into the layer's 16-bit output. It is purely combinational and shared by all layers; the layer table supplies bias_shift, out_shift and apply_relu. It does five things in order:

- sign-extend the bias to 48 bits and shift it left by bias_shift, so it lines up with the accumulator's fractional bits;
- add it to the sum;
- shift right by out_shift with an **arithmetic** shift, which rounds toward minus infinity exactly like Python's integer >> (a disagreement here would mean the hardware and the golden model differ on every negative activation);
- clamp (saturate) to −32768 … 32767 instead of letting the value wrap around, and raise the diagnostic flag overflow if that actually changed the value;
- if apply_relu is 1, replace a negative value by 0.

Port	Dir	Width	Meaning
acc_in	in	48	Finished sum from mac_unit.
bias_in	in	16	Bias of this output channel.
bias_shift, out_shift	in	5 each	Per-layer shifts (12/16, 10/15, 10/16, 9/14).
apply_relu	in	1	1 for conv1, conv2, fc1; 0 for fc2.
result	out	16	Saturated, ReLU-ed activation.
overflow	out	1	1 if saturation clamped this value. Diagnostic only; not used by ecg_top.

Table 13. Worked examples (from the testbench; every expected value was computed in plain Python integers first).

Case	acc_in	bias	Shifts	Sum after bias	After >>> shift	result	overflow
conv1, positive	80,000,000	−3,669	12 / 16	64,971,776	991	991	0
conv2, ReLU zeroes	−600,000,000	2,000	10 / 15	−597,952,000	−18,249	0	0
fc2, no ReLU, in range	−6,000,000	2,000	9 / 14	−4,976,000	−304	−304	0
fc2, no ReLU, saturates	−600,000,000	2,000	9 / 14	−598,976,000	−36,559	−32,768	1

The last row shows why the shifts matter: the same sum would have fitted comfortably in the original Q2.14 plan, but under the final shifts it genuinely saturates. **Note:** the comment block at the top of bias_rescale.v still describes the original plan (Q2.14 outputs, shifts 12 and 15). The code and the layer table in ecg_top.v are correct and carry the final per-layer values above; only the comment is stale (Section 9).

Vivado. 106 LUTs, no flip-flops: the largest single block by LUT count, because of the variable shifters, the 48-bit add and the clamp. The slowest path of the whole design starts at the

layer-number register and ends at the write-data pin of buf_b, so it passes through this logic and the pooling comparator (Section 7).

Verification. Nine directed cases (the four above plus conv1 filter 0's real values, fc1 positive, conv2 positive and a positive and a negative forced overflow); three deliberate faults were caught (skipping the clamp on the ReLU path, an off-by-one shift, and an overflow flag stuck low while the result stayed correct, a reminder that a test checking only the main output can miss a fault in a diagnostic signal). In ecg_top_tb every finished value of every layer (4,066 per beat) is compared at the output of this module, and the number of overflow flags per beat must equal the Python model's count.

5.8 max_pool and pool_pair: pooling one number at a time

max_pool (30 lines) is the primitive: `max_val = (a > b) ? a : b`, a signed 16-bit comparison and a 2:1 multiplexer (b is returned on a tie). It is combinational. In the real pipeline it only sees non-negative numbers, since it runs after ReLU, but it makes no such assumption.

pool_pair (67 lines) exists because the single MAC finishes the two neighbours of a pair one after the other: when the first appears, its partner does not exist yet. pool_pair parks the first number in one 16-bit register and, when the second arrives, passes both to max_pool and hands on the winner. The unpooled outputs are never stored anywhere.

Table 14. Behaviour of pool_pair.

in_valid	pos_odd	What happens
1	0	First of a pair: held <= in_val. out_valid = 0 (nothing to write yet).
1	1	Second of a pair: out_val = larger of held and in_val; out_valid = 1 (write now).
0	x	Nothing; held keeps its value for as long as it takes.

pos_odd is the lowest bit of the output's position number, so every number says by itself whether it is first or second: there is no toggle flag inside the module that could fall out of step. The pooled position is position[MSB:1]. The number of positions per filter must be even (256 and 124 are). held has no reset; it is never used before its first load.

Verification. max_pool: ten hand-computed cases including the full-scale boundaries in both operand orders (32767 against −32768, and both at −32768); three deliberate faults caught (comparison inverted, second input ignored, 'signed' dropped from one input). pool_pair: 16,384 pooled conv1 values (16 beats × 8 filters × 128), 15,872 pooled conv2 values (16 × 16 × 62) and 20,020 directed and random pairs compared with the key; 717,722 checks in all, 0 errors.

5.9 decision: the final comparison

decision (20 lines): `abnormal = (score1 > score0)`, a signed compare of the two Q6.10 scores. A tie means Normal. This equals the Python rule and the softmax rule p1 > 0.5, so no softmax or division is needed in hardware. It is combinational.

Verification. The testbench compares with a 32-bit integer comparison, a different method from the signed 16-bit compare in the module: the 16 key beats (including beat 9, an exact tie that must give Normal); every one of the 65,536 possible score0 values against 12 partners each (itself, ±1, the extremes, small values and random ones); and 1,000,000 random pairs, some forced equal. 1,720,910 checks, 0 errors.

5.10 ecg_top: putting it together

ecg_top (442 lines, 129 of them comment-only) is the four layer-stage files merged into one, with the settings that differ between the layers chosen by the layer number. It contains no new arithmetic. Its own logic is the glue:

Table 15. Instances inside ecg_top.

Instance	Module	Role
seq	layer_seq	Which layer, start_layer, beat_busy, beat_done.
ctrl	layer_ctrl	Counts the taps of one answer. Sees the undelayed signals.
agen	addr_gen	Addresses. init = start_layer, step = enable, next_output = done.
in_buf, buf_a, buf_b, buf_c	data_buffer × 4	Depths 260 / 1,024 / 992 / 32; all four get the same read address.
wrom	weight_rom	Read address = w_base + weight_addr.
brom	bias_rom	Read address = b_base + filter_d.
mac	mac_unit	Gets sample, weight, clear_d, enable_d.
rescale	bias_rescale	Bias, shift, clamp, ReLU (apply_relu = not fc2).
pool (and its max_pool 'compare')	pool_pair	Conv layers only: in_valid = done_d and layer is 0 or 1.
dec	decision	Compares the two score registers.

- **Starting an answer.** A flag `active` is set by start_layer and cleared in the done clock of the layer's last answer. While it is set and layer_ctrl is idle, a new answer is started: `start = active && !busy && !done`.
- **Choosing the data.** The layer number selects which buffer feeds the MAC: in_buf, buf_a, buf_b or buf_c for layers 0 to 3.
- **Storing the result.** Only the buffer of the running layer is written: buf_a when pool_valid and layer 0, buf_b when pool_valid and layer 1, buf_c in the done_d clock of layer 2, and score0 or score1 in the done_d clock of layer 3, chosen by the low bit of filter_d.
- **Layer finished.** layer_done = a register holding (done_d and last_d): it comes one clock after the last result was written, and the sequencer needs one more clock before it starts the next layer, so the next layer always starts on completely written data.
- **Address ranges** were checked against the largest address each layer uses: in_buf 259, buf_a 1,023, buf_b 991, buf_c 31, weight_rom 32,487.

The ecg_top header notes one thing to watch: the path from mac_unit through bias_rescale and pooling into a buffer is a single clock long, and the shifter that is now chosen by the layer makes it longer than in the single-layer files. At 50 MHz it meets timing with 5.4 ns to spare, so the suggested remedy (a register after bias_rescale, with filter_d, pos_d and done_d delayed one more clock) is not needed.

5.11 The four stage files

Before ecg_top existed, each layer was built and proved on its own from the verified modules. conv1_stage.v, conv2_stage.v, fc1_stage.v and fc2_stage.v each contain the wiring, delay registers and write address of one layer and no new maths. They stay in the project as scaffolding and as the reference from which ecg_top was assembled; they are not synthesized. Each reads its input from the answer key (the previous layer's expected output) so that a layer can be tested independently of the others.

Table 16. The stage files and their tests.

File (lines)	Layer	Test (lines)	What the testbench compared	Errors
conv1_stage (282)	input → conv1 → ReLU → pool → buf_a	314	16,384 buffer-A words (16 beats × 1,024); 40,960 values before pooling; 20,480 write-once checks; 286,816 protocol checks; 20 runs	0
conv2_stage (277)	buf_a → conv2 → ReLU → pool → buf_b	325	15,872 buffer-B words; 39,680 values before pooling; 19,840 write-once checks; 1,666,656 protocol checks; 20 runs	0
fc1_stage (256)	buf_b → fc1 → ReLU → buf_c	337	512 buffer-C words; 640 neuron values; 7 saturation flags equal to the key; 636,256 protocol checks; 20 runs	0
fc2_stage (266)	buf_c → fc2 → scores → decision	459	16 key beats plus 2,000 random vectors (2,420 runs, 4,840 scores); 253 saturation flags equal to the testbench's own calculation; 103,633 'results hold' clocks; 176,256 protocol checks	0

6. Verification

6.1 Strategy

The design was verified bottom-up, and every level compares against numbers that were produced *independently* of the Verilog: the Python integer model of the quantised network (the golden model) and, where possible, plain integer arithmetic worked out separately in the testbench itself.

Table 17. The verification levels.

Level	What is tested	Against
1. Modules	Each of the 12 modules alone, with its own testbench.	Hand-computed cases, Python checksums, independent models inside the testbench.
2. Layers	conv1, conv2, fc1 and fc2 each wired from the verified modules (stage files).	Python answer key: the layer's input and expected output for 16 real beats.
3. Whole chip	ecg_top: 260 samples in, scores and decision out, with resets, restarts and stray inputs.	Answer key: every value of every layer, the scores, the decision, the exact clock counts.
4. More beats	The same whole-chip test with three further sets of 16 randomly drawn test beats.	Answer keys made the same way (seeds 4242, 777, 31337).
5. Fault injection	Deliberately broken copies of the design, to show that the tests can fail.	The same testbenches: every broken copy should be caught.
6. Gate level	The synthesized netlist (Yosys), simulated with Xilinx cell models.	The same answer key.
7. Lint	Verilator 5.020 with all warnings on, on the 12 modules.	No warnings (Section 6.5).

6.2 The answer key

The key holds 16 real test beats and, for each, the expected output of every layer from the validated fixed-point model: 2,048 + 1,024 + 1,984 + 992 + 32 + 2 values plus the decision. The beats were **chosen to stress the hardware**, not to represent the model's accuracy: they include the largest activations of each layer, beats where fc1 saturates, two beats that the model gets wrong, and the two closest calls to the decision boundary, one of them an exact tie. Seven of the 16 are therefore misclassified by the model itself, on purpose: the hardware must reproduce the model's answer whether it is right or wrong.

Table 18. The 16 beats of the answer key (scores in Q6.10, shown as decimals). Test beat = index in the 24,526-beat test set.

#	Test beat	True	Model	score0	score1	fc1 clamps	Why chosen
0	19138	Nor	Nor	1.4893	−1.3457	0	typical Normal
1	12443	Abn	Abn	−6.8301	8.2842	0	typical Abnormal
2	5748	Nor	Abn	0.5811	3.6475	4	fc1 saturates (4 values)
3	5681	Nor	Abn	0.6689	2.2168	1	fc1 saturates (1 value)
4	5208	Nor	Abn	−3.3633	0.0381	0	largest conv1 activation
5	5747	Nor	Abn	−0.8926	6.0771	1	largest conv2 activation
6	5730	Nor	Abn	−3.4277	0.2012	0	largest fc1 value before clamping
7	12676	Nor	Abn	−1.6709	4.1729	0	false positive of the model
8	19716	Abn	Nor	1.0352	−0.6436	0	false negative of the model
9	14558	Nor	Nor	0.6133	0.6133	0	exact tie: must give Normal
10	13914	Nor	Nor	0.5889	0.5879	0	second-closest call
11	7223	Nor	Nor	2.3125	−1.9893	0	typical Normal
12	3520	Abn	Abn	−1.7520	3.2109	0	typical Abnormal
13	5413	Nor	Nor	2.0430	−1.3330	0	typical Normal
14	3887	Abn	Abn	−1.8076	2.1475	0	typical Abnormal
15	13487	Nor	Nor	1.8369	−0.9131	0	typical Normal

6.3 Whole-chip test (ecg_top_tb)

For each of the 16 beats the testbench loads the 260 samples into the input buffer, waits a random 0 to 5 clocks, pulses start_beat and waits for beat_done. On the beat_done clock, score0, score1 and abnormal must equal the key, and must then stay unchanged until the next beat. It also does the following:

- Some beats are loaded with random idle clocks between samples; beats 3, 7, 11 and 15 are run twice in a row, the second run starting on the very clock beat_done is seen (fastest possible restart).
- Beats 5, 6, 9, 10 and 14 are first interrupted by a **reset** in the middle of the beat: during conv1, conv2, exactly at the end of conv1, during fc1 and during fc2. The run after the reset must still be exact, and an interrupted run must not produce a beat_done.

- Beats 2, 7 and 12 receive **stray start_beat pulses** during the beat, including in the clocks where a layer finishes and the next one starts. They must be ignored and must not change a single result or clock count.
- Buffers A, B, C and both score registers are filled with **garbage** before every run, so a word that is never written cannot pass because it already held the right number.
- Every finished value of every layer (4,066 per beat) is compared at the output of bias_rescale, before pooling, and each must appear exactly once; then all 1,024 + 992 + 32 buffer words are compared, and every buffer address must have been written exactly once per beat.
- The saturation flag must be raised exactly as often as the key says (beat 2: 4 values, beat 3: 1, beat 5: 1, all others 0).
- The layers must run in order 0, 1, 2, 3, each started by one start_layer pulse, and layer_done must come at exactly 14,339 / 97,670 / 129,481 / 129,552 clocks; beat_done exactly 129,552 clocks after start_beat; beat_busy 0 before, 1 throughout, 0 on the beat_done clock.
- score0 and score1 may change only in the clock after fc2's own neuron 0 or neuron 1 is finished.

Table 19. Result of the whole-chip test on the 16-beat key (29.06 ms of simulated time).

Check	Count
Runs of the network / beat_done pulses	20 / 20
Results compared on beat_done (scores and decision)	20 runs
Finished values compared at bias_rescale	81,320 (20 runs × 4,066)
Buffer words compared with the key	40,960 (20 runs × 2,048)
Buffer address written-once checks	41,600
Clocks checked for 'results hold'	5,937
Saturation flags seen (must equal the key's count on every run)	7
Score-register changes seen (any change at another clock is an error)	40
Protocol checks	2,591,096
Total errors	0

The same test was repeated with three further answer keys, each made from 16 test beats drawn at random (seeds 4242, 777 and 31337). Each made 20 runs, compared the same 81,320 finished values and 2,591,096 protocol checks, and finished with 0 errors. The keys for these sets contained no saturated values.

6.4 Fault injection

A test that passes is only meaningful if it can be shown to fail on a wrong design. For ecg_top.v, 111 broken copies were made, each with one change, and run against the whole-chip testbench (6 min 43 s of wall-clock time). 40 changed one number of the layer table (every setting of every layer, +1); 71 broke the wiring: delays removed or added, write addresses changed, buffers swapped, ReLU and shifts altered, score registers mixed up, and more.

Table 20. Fault-injection campaigns.

Campaign	Broken copies	Caught	Not caught
ecg_top.v: layer table (10 settings × 4 layers, +1 each)	40	35	5
ecg_top.v: wiring and delays	71	65	6
ecg_top.v, total	111	100	11
layer_seq.v	24	24	0
addr_gen.v (earlier report)	8	8	0
layer_ctrl.v (earlier report)	3	3	0
bias_rescale.v (earlier report)	3	3	0
max_pool.v (earlier report)	3	3	0

The 11 copies that were **not** caught were each examined. None of them can change a value, a clock count or an output under the design's rules, so they are reported as functionally equivalent changes rather than as holes in the test. This is reasoned from the logic, not formally proven.

Table 21. The 11 undetected variants.

Change that went undetected	Why it cannot matter
num_channels of fc1 or fc2 +1 (2 copies)	With one row and row_len 0, addr_gen's channel counter adds nothing to the address, and layer_ctrl, not addr_gen, ends each answer.
row_len of conv1, fc1 or fc2 +1 (3 copies)	row_len is only used when there is more than one input row, which is conv2 alone (the conv2 change is caught).
clear_acc to the MAC not delayed	The clear then arrives one clock early, in a clock where enable_d is still 0, so the sum is cleared and the first product is added in the next clock: the same result. The previous sum was read at least two clocks earlier.
MAC enable stuck at 1	The accumulator also adds in idle clocks, but only after the finished sum has been read, and the first tap of every answer starts a fresh sum. Same results; it would only waste power.
Sample and weight swapped at the MAC	Multiplication is commutative.
Pooling also enabled in the fc layers	The buffer write enables are gated by the layer number anyway.
addr_gen init tied off	addr_gen wraps by itself at the end of every layer, and reset restarts it. init is a safety net.
addr_gen next_output given the delayed done	The position and filter counters move on one clock later but still before the next answer's first address is needed.

No separate fault-injection campaign was run for pool_pair, data_buffer, decision, weight_rom, bias_rom and mac_unit. Their wiring is covered by the ecg_top campaign, and their internals by their own testbenches (Section 5), but that is a weaker argument than fault injection and is stated here as a gap.

6.5 Gate level and lint

Gate level. The whole design was synthesized with Yosys into a flat netlist of 1,171 cells (including 32 RAMB18E1 and 1 DSP48E1) and simulated with Icarus Verilog, using Yosys's own Xilinx cell models and a hand-written behavioural model of the RAMB18E1 block RAM. The ecg_top testbench ran on beats 0 (typical Normal), 2 (fc1 saturates, 4 values) and 9 (exact tie): each reproduced the key exactly with 129,555 protocol checks and 0 errors (8 to 11 minutes per beat). A netlist corrupted on purpose was caught at the first score (score0 read 8000, expected 05F5). This shows that the synthesized logic, not only the RTL, does what the key says. It is the *Yosys* netlist; the netlist that Vivado produced has not been simulated (Section 9).

Lint. Verilator 5.020 with `-Wall` on all 12 modules reports no warnings, apart from a missing newline at the end of mac_unit.v, which is purely cosmetic. ecg_top.v contains explicit lint waivers for four signals that are deliberately unused (term_idx, the top address bits, the top position bit and the overflow diagnostic).

7. Synthesis and implementation in Vivado

7.1 Setup

Tool: Vivado 2026.1. Chip: xc7z020clg400-1, the Zynq-7020 used on PYNQ-Z1 and PYNQ-Z2 boards. Clock constraint: 20 ns (50 MHz). The design was synthesized and implemented **out of context**: on its own, with no board, no pins and no processor around it, and an ideal clock. The flow is run by two scripts, run_synth.tcl (synth_design) and run_impl.tcl (opt_design, place_design, phys_opt_design, route_design), which write the reports and a short summary and do not change any design file. The memory files weights.hex and biases.hex are read with $readmemh during synthesis and are therefore part of the result.

7.2 Results

Table 22. Vivado results for ecg_top (xc7z020clg400-1, 50 MHz target, out of context).

	After synthesis	After place and route	Available
LUTs	398 (0.75 %)	394 (0.74 %)	53,200
of which LUT as memory	–	12 (0.07 %)	17,400
Flip-flops	213 (0.20 %)	213 (0.20 %)	106,400
Block RAM tiles	17.5 (12.50 %)	17.5 (12.50 %)	140
DSP blocks	2 (0.91 %)	2 (0.91 %)	220
Slices	–	128 (0.96 %)	13,300
Setup slack at 20 ns	+6.622 ns (estimate)	+5.424 ns	
Hold slack	–	+0.108 ns	
Failing endpoints (setup / hold)	–	0 / 0	
Slowest path	13.053 ns, 20 logic levels	14.049 ns, 20 logic levels (logic 5.319 + wires 8.730)	
Errors / critical warnings / warnings	0 / 0 / 2	0 / 0 / 34	
Routing	–	all 643 routable nets routed, 0 routing errors	

	After synthesis	After place and route	Available
Estimated power	–	0.126 W total, 0.022 W dynamic (vectorless)	

The slowest path after routing runs from the layer-number register (seq/layer_r_reg[0]) to the write-data pin of buf_b. 62 % of its 14.049 ns is wire delay, and the path has a high-fanout net (143 loads). A path of 20 logic levels is dominated by wiring in a design this spread over the chip, which is expected. The setup slack of 5.424 ns means the design could be clocked at roughly 69 MHz (a period of about 14.6 ns), but it does not need to be: at 50 MHz it takes 2.59 ms per beat against a budget of 300 ms.

All warnings are consequences of out-of-context mode: the clock port has no HD.CLK_SRC property (it is not tied to a pin), 28 warnings say that rst_n has no partial-route location, 2 say that not all connectivity DRCs could be run in out-of-context mode, and 1 says that tiles could not be split because device files are unavailable. In synthesis, the register pos_d was trimmed from 9 to 8 bits because its top bit is never used (positions go from 0 to 255).

7.3 Where the area goes

Table 23. Resources by block after place and route (hierarchy view). Slices per block overlap, so they do not add up to 128.

Block (Vivado hierarchy)	LUTs	Flip-flops	Slices	DSP	Remarks
bias_rescale (rescale)	106	0	49	0	Combinational: shifters, 48-bit add, clamp, ReLU.
addr_gen (agen)	81	62	35	0	Counters and running base addresses.
mac_unit (mac)	80	48	24	2	48-bit accumulator in logic; multiplier in 2 DSP48E1.
layer_seq (seq)	78	5	32	0	Larger than a three-state machine needs; reason not investigated.
rest of ecg_top (by subtraction)	49	98	–	0	layer_ctrl, pool_pair, decision, delay registers, score registers, buf_c, glue.
Total	394	213	128	2	382 LUTs as logic + 12 as memory; 17.5 block-RAM tiles.

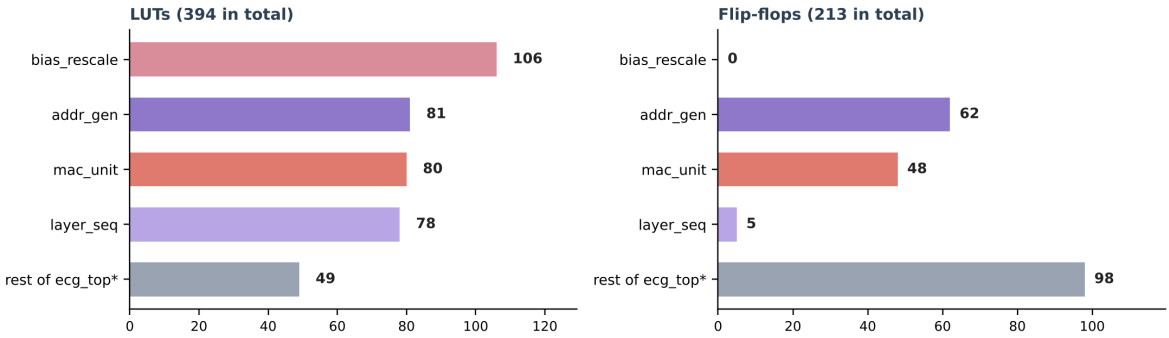


Figure 6. LUTs and flip-flops by block. *rest of ecg_top = the total minus the four blocks Vivado lists separately; it includes layer_ctrl, which Vivado did not list as its own line.

Memory. The 17.5 block-RAM tiles are 35 half-tiles (RAMB18). The weights alone need 32 of them (32,488 words of 16 bits in 1,024-word half-tiles), so the weight storage is the largest resource cost of the whole design, 63.5 KiB, and the remaining roughly 3 half-tiles hold the input buffer and the two large activation buffers. The 12 LUTs used as memory are consistent with the 32 × 16 buffer C in distributed RAM. This split is inferred from the sizes, not read from the per-instance report.

7.4 Open-source versus Vivado numbers

Table 24. The same design through two tools. The Yosys netlist is the one used for the gate-level simulation; Vivado is the tool for the target chip.

	Yosys (open source)	Vivado synthesis	Vivado place and route
LUTs	568 (LUT2 to LUT6)	398	394
Flip-flops	219	213	213
DSP48E1	1	2	2
Block RAM	32 × RAMB18	17.5 tiles = 35 × RAMB18	17.5 tiles
Carry cells	73 CARRY4	–	–

8. Performance

Table 25. Performance summary.

Quantity	Value	How obtained
Clocks per beat	129,552	Counted in simulation and predicted by formula (Section 4.1).
Time per beat at 50 MHz	2.591 ms	129,552 × 20 ns.
Beats per second, back to back	385.9	1 / 2.591 ms.
Real-time need	3.3 beats/s (200 bpm, 300 ms per beat)	Fast but still realistic heart rate.
Headroom	about 116 ×	300 ms / 2.591 ms.
Multiplier busy	93.7 % of clocks	121,408 useful multiplies in 129,552 clocks.
Largest layer	conv2: 64.3 % of the time	83,330 of 129,552 clocks.
Dynamic energy per beat (estimate)	about 57 µJ	0.022 W dynamic × 2.591 ms; vectorless estimate, not measured.

The design is deliberately small and slow rather than large and fast: one multiplier does everything, in 2.6 ms, which is already two orders of magnitude faster than a heartbeat requires. If a higher rate were ever needed, the obvious step would be a second MAC for conv2, the layer that dominates the time; this is not needed for the project's goal.

9. Known limitations and open items

Table 26. Open items.

Item	What it means	Next step
Out-of-context results	Timing and power were obtained with the design alone, an ideal clock and no I/O. Hold slack (+0.108 ns) in particular is not a sign-off figure.	Re-run with the real clock, pins and the processor system once the wrapper exists.
Routed netlist not simulated	Functional simulation was done on the RTL and on the Yosys netlist, both in Icarus Verilog. The Vivado simulator hung when it was tried on the development PC (cause not resolved), so Vivado's own simulator was not used and the netlist that Vivado produced was not simulated.	Try the Vivado simulator on another machine, or simulate Vivado's netlist in Icarus with the Xilinx cell library.

Item	What it means	Next step
No bitstream, nothing run on a board	No PYNQ wrapper exists yet. The design's interface is the simple one in Section 2.1.	Next project step: wrapper and notebook that replays stored beats.
Asynchronous reset	rst_n is asynchronous, as in all modules. Releasing it asynchronously on a real board can break timing.	Synchronise the release of reset in the wrapper.
Stale comment	The comment block at the top of bias_rescale.v still describes the original Q2.14 plan (shifts 12 and 15). The code and the layer table are correct.	Update the comment; no functional change.
Input protocol is not protected	Writing samples during a beat corrupts that beat; in_waddr beyond 259 is not guarded.	The wrapper must follow the usage sequence of Section 2.1.
2 DSP blocks instead of 1	Vivado split the multiplier over two DSP48E1 and built the accumulator in logic.	Optional: restructure mac_unit for a single DSP; no benefit needed now.
Power	0.022 W dynamic is a vectorless estimate.	Measure on the board, or run a simulation-driven estimate.
Coverage gaps in fault injection	No separate campaign for pool_pair, data_buffer, decision, weight_rom, bias_rom, mac_unit (Section 6.4).	Optional.
Cosmetic	run_impl.tcl prints 'n/a unrouted nets' in its summary; mac_unit.v lacks a final newline; addr_gen_two.v in the project folder is an unused experiment.	Clean up when convenient.

10. How to reproduce the results

All commands are run from the project folder, which must contain the 12 RTL files, weights.hex, biases.hex and (for the testbenches) the answer_key folder.

Whole-chip simulation (about one minute):

```
iverilog -o ecg_top_sim ecg_top.v layer_seq.v decision.v addr_gen.v layer_ctrl.v mac_unit.v bias_rescale.v max_pool.v \
        pool_pair.v bias_rom.v weight_rom.v data_buffer.v ecg_top_tb.v
vvp ecg_top_sim
```

The last lines must read `total errors : 0` and `ALL TESTS PASSED`. Options: `-DNBEATS=4` for fewer beats, `-DFIRST=9` to start at beat 9, `-DSTOP_ON_FAIL` to stop at the first error. Every other testbench is compiled the same way (the compile line is in its header).

Vivado (Windows, one command per line):

```
& "D:\2026.1\Vivado\bin\vivado.bat" -mode batch -nojournal -log synth.log -source run_synth.tcl
& "D:\2026.1\Vivado\bin\vivado.bat" -mode batch -nojournal -log impl.log -source run_impl.tcl
```

The scripts write synth_out and impl_out with the reports and a short summary file each. The path to vivado.bat depends on where Vivado is installed.

Appendix A. File inventory

Table 27. Project files.

File	Lines	Purpose
ecg_top.v	442	Top level (synthesized).
layer_seq.v, layer_ctrl.v, addr_gen.v	93, 66, 137	Control (synthesized).
weight_rom.v, bias_rom.v, data_buffer.v	41, 35, 55	Memories (synthesized).
mac_unit.v, bias_rescale.v, max_pool.v, pool_pair.v, decision.v	31, 63, 30, 67, 20	Arithmetic and decision (synthesized).
conv1_stage.v, conv2_stage.v, fc1_stage.v, fc2_stage.v	282, 277, 256, 266	One layer each; verification scaffolding.
ecg_top_tb.v	523	Whole-chip testbench.
layer_seq_tb.v, layer_ctrl_tb.v, addr_gen_tb.v	212, 120, 208	Unit testbenches (control).
weight_rom_tb.v, bias_rom_tb.v, data_buffer_tb.v	192, 112, 475	Unit testbenches (memories).
mac_unit_tb.v, bias_rescale_tb.v, max_pool_tb.v, pool_pair_tb.v, decision_tb.v	41, 78, 48, 227, 102	Unit testbenches (arithmetic).
conv1_stage_tb.v, conv2_stage_tb.v, fc1_stage_tb.v, fc2_stage_tb.v	314, 325, 337, 459	Layer testbenches.
weights.hex, biases.hex	32,488 / 58	ROM contents, one 4-digit hex number per line.
answer_key/ (10 files)	–	The 16-beat answer key; answer_key_info.txt describes the layout.
export_rom_files.py, make_answer_key.py, fixed_point_forward_v2.py	–	Python: ROM export, answer key, fixed-point reference model.
run_synth.tcl, run_impl.tcl, run_sim.tcl	–	Vivado batch scripts (synthesis, implementation, simulation).

Appendix B. Source listing of the 12 synthesizable modules

Printed exactly as in the project folder (ecg_top.v first, then the control modules, arithmetic, memories). Line numbers are added. The comments in the files are part of the design documentation.

B.1 ecg_top.v (442 lines)

```
  1 // ecg_top.v
  2 // The whole ECG classifier: one heartbeat in (260 samples), one decision out (Normal / Abnormal).
  3 //
  4 //   samples -> conv1 -> ReLU -> pool -> conv2 -> ReLU -> pool -> fc1 -> ReLU -> fc2 -> decision
  5 //
  6 // ONE shared multiply-accumulate unit (mac_unit) does all the arithmetic of all four layers, one after
  7 // the other. The layer sequencer (layer_seq) says which layer is running; a small table in this file
  8 // turns the layer number into that layer's settings (how many filters, where its weights start, ...).
  9 // Everything else is the wiring that was already built and tested layer by layer in conv1_stage.v,
 10 // conv2_stage.v, fc1_stage.v and fc2_stage.v - this file is those four files merged into one,
 11 // with the settings that differ between the layers chosen by the layer number.
 12 //
 13 // ------------------------------------------------------------------------
 14 // HOW TO USE IT
 15 // ------------------------------------------------------------------------
 16 //   1. Load the beat: for each of the 260 samples (Q4.12, signed) set in_we = 1, in_waddr = the sample
 17 //      number (0..259), in_wdata = the value, one value per clock. Do NOT write during a beat.
 18 //   2. Pulse start_beat for one clock (while beat_busy = 0).
 19 //   3. Wait for beat_done (one-clock pulse). From that clock on score0 (Normal), score1 (Abnormal) and
 20 //      abnormal are final. They stay unchanged until the next beat reaches its last layer.
 21 //   One beat takes exactly 129,552 clocks from the start_beat clock to the beat_done clock
 22 //   (conv1 14,338 + conv2 83,330 + fc1 31,810 + fc2 70 clocks of work, plus 1 clock to start the
 23 //   first layer and 1 clock between layers = 4 clocks), i.e. 2.59 ms at 50 MHz.
 24 //
 25 // ------------------------------------------------------------------------
 26 // WHAT IS CONNECTED TO WHAT   (every connection below is made by name)
 27 // ------------------------------------------------------------------------
 28 //   layer_seq    : layer number (0 conv1, 1 conv2, 2 fc1, 3 fc2), start_layer pulse, beat_busy, beat_done
 29 //   layer table  : (below) the settings of the running layer: number of filters / positions / channels,
 30 //                  window length, row length, taps per answer, first weight address, first bias address,
 31 //                  the two shift amounts and whether ReLU is applied. Numbers from make_answer_key.py (LAYERS).
 32 //   layer_ctrl   : counts the taps of one answer. Gives clear_acc, enable, done.
 33 //   addr_gen     : turns "tap being used" into addresses. Takes layer_ctrl's enable (as step) and done
 34 //                  (as next_output), both UNdelayed. init = start_layer.
 35 //   in_buf       : data_buffer (260 words) = the beat's samples.            (read by conv1)
 36 //   buf_a        : data_buffer (1,024 words) = conv1 results, pooled.       (written by conv1, read by conv2)
 37 //   buf_b        : data_buffer (992 words) = conv2 results, pooled, already in flatten order.
 38 //                                                                           (written by conv2, read by fc1)
 39 //   buf_c        : data_buffer (32 words) = fc1 results.                    (written by fc1, read by fc2)
 40 //                  All four buffers get the same read address (addr_gen data_addr); the one that belongs
 41 //                  to the running layer is chosen by the layer number ('sample').
 42 //   weight_rom   : all 32,488 weights.  read address = the layer's first weight address + addr_gen weight_addr
 43 //   mac_unit     : sample * weight, summed.  Gets sample and weight (one clock after their address)
 44 //                  together with clear_acc / enable DELAYED by one clock.
 45 //   bias_rom     : biases.  address = the layer's first bias address + filter number, DELAYED by one clock
 46 //   bias_rescale : adds the bias, shifts, saturates to 16 bits, ReLU (not for fc2) - shifts chosen by the layer
 47 //   pool_pair    : the pooling holder, used by conv1 and conv2 only.
 48 //   buf_a / buf_b / buf_c write: see "WHERE THE RESULTS GO" below
 49 //   score regs   : fc2's two results (score0 = Normal, score1 = Abnormal)
 50 //   decision     : abnormal = score1 higher than score0 (a tie means Normal)
 51 //
 52 // ------------------------------------------------------------------------
 53 // THE ONE-CLOCK-DELAY RULE (same as in the layer files)
 54 // ------------------------------------------------------------------------
 55 //   weight_rom and data_buffer give their data ONE CLOCK AFTER the address, so everything the MAC
 56 //   side uses (mac_unit control, bias address, pooling, write addresses, score registers) is a copy
 57 //   of the layer_ctrl / addr_gen signal delayed by exactly one clock: clear_d, enable_d, done_d, last_d,
 58 //   filter_d, pos_d. addr_gen and layer_ctrl themselves are NOT delayed: they create the addresses.
 59 //   mac_unit registers its sum, so when done_d is high acc is the finished sum, and filter_d / pos_d still
 60 //   describe that finished output (addr_gen moves on at the end of the done clock).
 61 //
 62 // ------------------------------------------------------------------------
 63 // WHERE THE RESULTS GO   (all use done_d / the delayed filter and position)
 64 // ------------------------------------------------------------------------
```

```
//   conv1 (layer 0) : pool_pair -> buf_a at  filter * 128 + position / 2
//   conv2 (layer 1) : pool_pair -> buf_b at  filter * 62  + position / 2   (filter * 62 = filter * 64 - filter * 2)
//   fc1   (layer 2) : buf_c at  filter                (no pooling)
//   fc2   (layer 3) : score0 (neuron 0) or score1 (neuron 1)
//   Only the buffer of the running layer is written (the layer number gates the write enables).
//
// ------------------------------------------------------------------------
// WHEN THE NEXT LAYER MAY START
// ------------------------------------------------------------------------
//   A layer's layer_done comes one clock AFTER its last result was written, and the sequencer needs
//   one more clock before start_layer, so the next layer always starts on completely written data.
//
// Notes for whoever extends this (Vivado / timing):
//   - out_val of pool_pair and the result of bias_rescale are combinational in the same clock as the
//     finished sum: the path mac_unit -> bias_rescale -> max_pool -> buffer write is one clock long.
//     bias_rescale now shifts by an amount chosen by the layer (a variable shifter instead of fixed
//     wires), which makes this path longer than in the single-layer files. If it does not meet the
//     clock in Vivado, add a register after bias_rescale (and delay filter_d / pos_d / done_d one more clock).
//   - There is no reset on the memories; the buffers hold old data until a layer rewrites them.
//
module ecg_top (
    input  wire               clk,
    input  wire               rst_n,        // active low, asynchronous (like the other modules)

    // load the beat (260 samples, Q4.12)
    input  wire               in_we,
    input  wire [8:0]         in_waddr,     // 0 .. 259
    input  wire signed [15:0] in_wdata,

    // run the beat
    input  wire               start_beat,   // one-clock pulse
    output wire               beat_busy,    // 1 from the clock after start_beat until the clock before beat_done
    output wire               beat_done,    // one-clock pulse: score0, score1 and abnormal are final

    // results
    output wire signed [15:0] score0,       // class 0, Normal   (Q6.10)
    output wire signed [15:0] score1,       // class 1, Abnormal (Q6.10)
    output wire               abnormal      // 1 = Abnormal (score1 higher), 0 = Normal (also on a tie)
);

    // ------------------------------------------------------------------------
    // the sequencer
    // ------------------------------------------------------------------------
    wire [1:0] layer;            // 0 conv1, 1 conv2, 2 fc1, 3 fc2
    wire       start_layer;
    wire       layer_done;

    layer_seq seq (
        .clk        (clk),
        .rst_n      (rst_n),
        .start_beat (start_beat),
        .layer_done (layer_done),
        .layer      (layer),
        .start_layer(start_layer),
        .beat_busy  (beat_busy),
        .beat_done  (beat_done)
    );

    wire l_conv1 = (layer == 2'd0);
    wire l_conv2 = (layer == 2'd1);
    wire l_fc1   = (layer == 2'd2);
    wire l_fc2   = (layer == 2'd3);

    // ------------------------------------------------------------------------
    // the layer table: the settings of each layer in ONE place (from make_answer_key.py, LAYERS)
    //                    filters positions channels window row  taps  weights bias  bias  out   ReLU
    //                                                       len        start  start shift shift
    //   conv1 (layer 0)     8      256       1        5       0     5       0     0    12    16    yes
    //   conv2 (layer 1)    16      124       8        5     128    40      40     8    10    15    yes
    //   fc1   (layer 2)    32        1       1      992       0   992     680    24    10    16    yes
    //   fc2   (layer 3)     2        1       1       32       0    32   32424    56     9    14    no
    // ------------------------------------------------------------------------
    reg [5:0]  num_filters;
    reg [8:0]  num_positions;
    reg [3:0]  num_channels;
    reg [9:0]  kernel;
    reg [10:0] row_len;
    reg [10:0] num_terms;
    reg [14:0] w_base;
    reg [5:0]  b_base;
```

```
145     reg [4:0]  bias_shift;
146     reg [4:0]  out_shift;
147
148     always @(*) begin
149         case (layer)
150             2'd0: begin   // conv1
151                 num_filters   = 6'd8;
152                 num_positions = 9'd256;
153                 num_channels  = 4'd1;
154                 kernel        = 10'd5;
155                 row_len       = 11'd0;
156                 num_terms     = 11'd5;
157                 w_base        = 15'd0;
158                 b_base        = 6'd0;
159                 bias_shift    = 5'd12;
160                 out_shift     = 5'd16;
161             end
162             2'd1: begin   // conv2
163                 num_filters   = 6'd16;
164                 num_positions = 9'd124;
165                 num_channels  = 4'd8;
166                 kernel        = 10'd5;
167                 row_len       = 11'd128;
168                 num_terms     = 11'd40;
169                 w_base        = 15'd40;
170                 b_base        = 6'd8;
171                 bias_shift    = 5'd10;
172                 out_shift     = 5'd15;
173             end
174             2'd2: begin   // fc1
175                 num_filters   = 6'd32;
176                 num_positions = 9'd1;
177                 num_channels  = 4'd1;
178                 kernel        = 10'd992;
179                 row_len       = 11'd0;
180                 num_terms     = 11'd992;
181                 w_base        = 15'd680;
182                 b_base        = 6'd24;
183                 bias_shift    = 5'd10;
184                 out_shift     = 5'd16;
185             end
186             default: begin   // fc2
187                 num_filters   = 6'd2;
188                 num_positions = 9'd1;
189                 num_channels  = 4'd1;
190                 kernel        = 10'd32;
191                 row_len       = 11'd0;
192                 num_terms     = 11'd32;
193                 w_base        = 15'd32424;
194                 b_base        = 6'd56;
195                 bias_shift    = 5'd9;
196                 out_shift     = 5'd14;
197             end
198         endcase
199     end
200
201     wire apply_relu = !l_fc2;       // fc2 delivers the raw scores
202
203     // ------------------------------------------------------------------
204     // control: layer_ctrl + addr_gen   (these see the UNdelayed signals)
205     // ------------------------------------------------------------------
206     // (lint: term_idx, the top address bits and the top position bit are intentionally not used)
207     /* verilator lint_off UNUSEDSIGNAL */
208     wire [10:0] term_idx;
209     wire [10:0] data_addr;
210     wire [15:0] weight_addr;
211     /* verilator lint_on UNUSEDSIGNAL */
212     wire        clear_acc, enable, busy, done;
213     wire [5:0]  filter_idx;
214     wire [8:0]  position_idx;
215     wire        last_output;
216
217     // 'active' is high from start_layer until the very last answer's done clock. While it is
218     // high and layer_ctrl is idle, a new answer is started.
219     reg  active;
220     wire ctrl_idle = !busy && !done;
221     wire start     = active && ctrl_idle;
222
223     always @(posedge clk or negedge rst_n) begin
224         if (!rst_n)
```

```
225             active <= 1'b0;
226         else if (start_layer)
227             active <= 1'b1;
228         else if (done && last_output)
229             active <= 1'b0;
230     end
231 
232     layer_ctrl #(.CNT_W(11)) ctrl (
233         .clk      (clk),
234         .rst_n    (rst_n),
235         .start    (start),
236         .num_terms(num_terms),
237         .clear_acc(clear_acc),
238         .enable   (enable),
239         .busy     (busy),
240         .done     (done),
241         .term_idx (term_idx)
242     );
243 
244     addr_gen agen (
245         .clk          (clk),
246         .rst_n        (rst_n),
247         .init         (start_layer),
248         .step         (enable),
249         .next_output  (done),
250         .num_filters  (num_filters),
251         .num_positions(num_positions),
252         .num_channels (num_channels),
253         .kernel       (kernel),
254         .row_len      (row_len),
255         .data_addr    (data_addr),
256         .weight_addr  (weight_addr),
257         .filter_idx   (filter_idx),
258         .position_idx (position_idx),
259         .last_output  (last_output)
260     );
261 
262     // ------------------------------------------------------------------------
263     // the one-clock delay registers
264     // ------------------------------------------------------------------------
265     reg        clear_d, enable_d, done_d, last_d;
266     /* verilator lint_off UNUSEDSIGNAL */
267     reg [8:0]  pos_d;          // bit 8 is not needed: positions go 0..255
268     /* verilator lint_on UNUSEDSIGNAL */
269     reg [5:0]  filter_d;
270 
271     always @(posedge clk or negedge rst_n) begin
272         if (!rst_n) begin
273             clear_d  <= 1'b0;
274             enable_d <= 1'b0;
275             done_d   <= 1'b0;
276             last_d   <= 1'b0;
277         end else begin
278             clear_d  <= clear_acc;
279             enable_d <= enable;
280             done_d   <= done;
281             last_d   <= last_output;
282         end
283     end
284 
285     always @(posedge clk) begin
286         filter_d <= filter_idx;
287         pos_d    <= position_idx;
288     end
289 
290     // ------------------------------------------------------------------------
291     // the four data buffers (each gives its data one clock after the address)
292     // ------------------------------------------------------------------------
293     wire signed [15:0] s_in, s_a, s_b, s_c;      // what each buffer delivers
294     wire signed [15:0] act;                      // finished output of the running layer (valid while done_d = 1)
295     wire               pool_valid;
296     wire signed [15:0] pool_val;
297 
298     // conv1 write address = filter * 128 + position / 2
299     wire [9:0] a_waddr = {filter_d[2:0], pos_d[7:1]};
300 
301     // conv2 write address = filter * 62 + position / 2, with filter * 62 = filter * 64 - filter * 2
302     wire [9:0] filter_x62 = {filter_d[3:0], 6'b000000} - {5'b00000, filter_d[3:0], 1'b0};
303     wire [9:0] b_waddr    = filter_x62 + {4'b0000, pos_d[6:1]};          // largest: 15 * 62 + 61 = 991
304 
```

```
    data_buffer #(.DEPTH(260)) in_buf (
        .clk  (clk),
        .we   (in_we),
        .waddr(in_waddr),
        .wdata(in_wdata),
        .raddr(data_addr[8:0]),        // largest address used: 255 + 4 = 259
        .rdata(s_in)
    );

    data_buffer #(.DEPTH(1024)) buf_a (
        .clk  (clk),
        .we   (pool_valid && l_conv1),
        .waddr(a_waddr),
        .wdata(pool_val),
        .raddr(data_addr[9:0]),        // largest address used: 7 * 128 + 123 + 4 = 1023
        .rdata(s_a)
    );

    data_buffer #(.DEPTH(992)) buf_b (
        .clk  (clk),
        .we   (pool_valid && l_conv2),
        .waddr(b_waddr),
        .wdata(pool_val),
        .raddr(data_addr[9:0]),        // largest address used: 991
        .rdata(s_b)
    );

    data_buffer #(.DEPTH(32)) buf_c (
        .clk  (clk),
        .we   (done_d && l_fc1),
        .waddr(filter_d[4:0]),
        .wdata(act),
        .raddr(data_addr[4:0]),        // largest address used: 31
        .rdata(s_c)
    );

    // which buffer feeds the MAC: the one that belongs to the running layer
    reg signed [15:0] sample;
    always @(*) begin
        case (layer)
            2'd0:    sample = s_in;
            2'd1:    sample = s_a;
            2'd2:    sample = s_b;
            default: sample = s_c;
        endcase
    end

    // ------------------------------------------------------------------------
    // weights and biases
    // ------------------------------------------------------------------------
    wire signed [15:0] weight;
    wire [14:0] w_rom_addr = w_base + weight_addr[14:0];      // largest address: 32,424 + 63 = 32,487

    weight_rom wrom (
        .clk (clk),
        .addr(w_rom_addr),
        .data(weight)
    );

    wire signed [15:0] bias;

    bias_rom brom (
        .addr(b_base + filter_d),
        .data(bias)
    );

    // ------------------------------------------------------------------------
    // multiply-accumulate, bias, rescale (+ ReLU except for fc2)
    // ------------------------------------------------------------------------
    wire signed [47:0] acc;

    mac_unit mac (
        .clk      (clk),
        .rst_n    (rst_n),
        .clear_acc(clear_d),
        .enable   (enable_d),
        .data_in  (sample),
        .weight_in(weight),
        .acc_out  (acc)
    );
```

```
385
386     /* verilator lint_off UNUSEDSIGNAL */
387     wire act_ovf;                  // 1 = this output was clamped (diagnostic only, not used)
388     /* verilator lint_on UNUSEDSIGNAL */
389
390     bias_rescale rescale (
391         .acc_in    (acc),
392         .bias_in   (bias),
393         .bias_shift(bias_shift),
394         .out_shift (out_shift),
395         .apply_relu(apply_relu),
396         .result    (act),
397         .overflow  (act_ovf)
398     );
399
400     // ---------------------------------------------------------------------
401     // pooling holder (conv1 and conv2 only)
402     // ---------------------------------------------------------------------
403     pool_pair pool (
404         .clk      (clk),
405         .in_valid (done_d && !layer[1]),
406         .pos_odd  (pos_d[0]),
407         .in_val   (act),
408         .out_valid(pool_valid),
409         .out_val  (pool_val)
410     );
411
412     // ---------------------------------------------------------------------
413     // fc2's two results and the decision
414     // ---------------------------------------------------------------------
415     reg signed [15:0] score0_r, score1_r;
416
417     always @(posedge clk) begin
418         if (done_d && l_fc2 && !filter_d[0]) score0_r <= act;
419         if (done_d && l_fc2 &&  filter_d[0]) score1_r <= act;
420     end
421
422     assign score0 = score0_r;
423     assign score1 = score1_r;
424
425     decision dec (
426         .score0  (score0_r),
427         .score1  (score1_r),
428         .abnormal(abnormal)
429     );
430
431     // ---------------------------------------------------------------------
432     // layer finished: one clock after the last result of the layer was written
433     // ---------------------------------------------------------------------
434     reg done_r;
435     always @(posedge clk or negedge rst_n) begin
436         if (!rst_n) done_r <= 1'b0;
437         else        done_r <= done_d && last_d;
438     end
439
440     assign layer_done = done_r;
441
442 endmodule
```

B.2 layer_seq.v (93 lines)

```
 1 // layer_seq.v
 2 // The layer sequencer: runs the four layers of the network one after the other.
 3 //
 4 //     start_beat  ->  conv1  ->  conv2  ->  fc1  ->  fc2  ->  beat_done
 5 //     (layer 0)      (layer 1)  (layer 2)  (layer 3)
 6 //
 7 // It knows nothing about the maths. It only says WHICH layer is running (layer = 0, 1, 2, 3), gives
 8 // ONE clock pulse (start_layer) when a layer should begin, and waits for the datapath to say
 9 // layer_done. Then it moves on to the next layer.
10 //
11 // ---------------------------------------------------------------------------
12 // HOW IT WORKS
13 // ---------------------------------------------------------------------------
14 //   three states:   IDLE : nothing running, waiting for start_beat
15 //                   GO   : lasts exactly ONE clock; start_layer is high in this clock
16 //                   RUN  : a layer is running, waiting for layer_done
```

```
17 //
18 //   IDLE --start_beat--> GO (layer = 0) --1 clock--> RUN --layer_done--> GO (layer + 1) --> RUN ...
19 //   RUN with layer = 3 and layer_done --> IDLE   (beat_done is high in that same clock)
20 //
21 //   - start_layer comes exactly ONE clock after start_beat (for layer 0) and exactly ONE clock after
22 //     each layer_done (for layers 1, 2, 3). The layer number changes in the clock BEFORE start_layer,
23 //     so the layer's settings are stable when the layer starts.
24 //   - layer_done is only listened to in state RUN. A layer_done in IDLE or GO is ignored.
25 //   - start_beat is only accepted when no beat is running (IDLE) or in the very clock in which the
26 //     beat finishes (beat_done = 1, then the next beat starts at once). At any other time it is ignored.
27 //   - beat_done is high for ONE clock, in the same clock as the fourth layer_done.
28 //   - beat_busy is high from the clock after an accepted start_beat until the clock before beat_done;
29 //     it is 0 in the beat_done clock itself.
30 //   - rst_n (active low, asynchronous) puts everything back to IDLE, layer 0.
31 //   - Between beats the layer number stays at 3 (it is set back to 0 by the next start_beat).
32 //
33 module layer_seq (
34     input  wire       clk,
35     input  wire       rst_n,
36     input  wire       start_beat,    // pulse: run the four layers once
37     input  wire       layer_done,    // from the datapath: one-clock pulse when the running layer is finished
38
39     output wire [1:0] layer,         // 0 = conv1, 1 = conv2, 2 = fc1, 3 = fc2
40     output wire       start_layer,   // one-clock pulse: the datapath starts layer 'layer' now
41     output wire       beat_busy,     // a beat is running
42     output wire       beat_done      // one-clock pulse: all four layers are finished
43 );
44
45     localparam [1:0] S_IDLE = 2'd0,
46                      S_GO   = 2'd1,
47                      S_RUN  = 2'd2;
48
49     reg [1:0] state;
50     reg [1:0] layer_r;
51
52     wire last_layer = (layer_r == 2'd3);
53
54     // all four layers finished: the fourth layer_done, while running
55     assign beat_done = (state == S_RUN) && layer_done && last_layer;
56
57     // a new beat may start when nothing runs, or in the clock the old one finishes
58     wire accept = start_beat && ((state == S_IDLE) || beat_done);
59
60     always @(posedge clk or negedge rst_n) begin
61         if (!rst_n) begin
62             state   <= S_IDLE;
63             layer_r <= 2'd0;
64         end else if (accept) begin
65             state   <= S_GO;
66             layer_r <= 2'd0;
67         end else begin
68             case (state)
69                 S_GO: begin
70                     state <= S_RUN;                 // GO lasts one clock
71                 end
72                 S_RUN: begin
73                     if (layer_done) begin
74                         if (last_layer) begin
75                             state <= S_IDLE;         // beat finished
76                         end else begin
77                             layer_r <= layer_r + 2'd1;
78                             state   <= S_GO;         // next layer starts in the next clock
79                         end
80                     end
81                 end
82                 default: begin
83                     state <= S_IDLE;                // IDLE stays idle; the unused code 3 falls back to IDLE
84                 end
85             endcase
86         end
87     end
88
89     assign layer       = layer_r;
90     assign start_layer = (state == S_GO);
91     assign beat_busy   = (state != S_IDLE) && !beat_done;
92
93 endmodule
```

B.3 layer_ctrl.v (66 lines)

```
 1  // layer_ctrl.v
 2  // Sequences the single MAC unit through the N terms of ONE output value
 3  // (one conv filter position, or one fully-connected neuron).
 4  //
 5  //   start  : pulse for one cycle (while IDLE) to begin an output value
 6  //   num_terms : how many multiply-accumulates this output needs (must be >= 1)
 7  //               conv1 = 5, conv2 = 40, fc1 = 992, fc2 = 32
 8  //   term_idx  : which term is being fed this cycle (0 .. num_terms-1);
 9  //               later this becomes the address for the data / weight memories
10  //   clear_acc : high on the FIRST term only -> MAC starts a fresh sum
11  //   enable    : high on every term -> MAC accumulates
12  //   done      : high for ONE cycle after the last term; acc_out is final then
13  //
14  module layer_ctrl #(
15      parameter CNT_W = 11                    // 11 bits counts up to 2047 (fc1 needs 992)
16  )(
17      input  wire             clk,
18      input  wire             rst_n,
19      input  wire             start,
20      input  wire [CNT_W-1:0] num_terms,
21      output wire             clear_acc,
22      output wire             enable,
23      output wire             busy,
24      output wire             done,
25      output wire [CNT_W-1:0] term_idx
26  );
27
28      localparam [1:0] IDLE = 2'd0,
29                       RUN  = 2'd1,
30                       DONE = 2'd2;
31
32      reg [1:0]       state;
33      reg [CNT_W-1:0] count;
34
35      always @(posedge clk or negedge rst_n) begin
36          if (!rst_n) begin
37              state <= IDLE;
38              count <= 0;
39          end else begin
40              case (state)
41                  IDLE: begin
42                      count <= 0;
43                      if (start) state <= RUN;
44                  end
45                  RUN: begin
46                      if (count == num_terms - 1)
47                          state <= DONE;          // this edge also registers the last term
48                      else
49                          count <= count + 1;
50                  end
51                  DONE: begin
52                      state <= IDLE;
53                  end
54                  default: state <= IDLE;
55              endcase
56          end
57      end
58
59      // Outputs depend only on state and count (Moore-style)
60      assign enable    = (state == RUN);
61      assign clear_acc = (state == RUN) && (count == 0);
62      assign busy      = (state == RUN);
63      assign done      = (state == DONE);
64      assign term_idx  = count;
65
66  endmodule
```

B.4 addr_gen.v (137 lines)

```
// addr_gen.v
// One address generator shared by conv1, conv2, fc1 and fc2.
//
// Each clock cycle that the MAC unit does a multiply, this module says WHICH
// box to read for the data (sample / activation) and WHICH box to read for
// the weight. The layer's shape comes in as inputs, so nothing here is
// specific to one layer (same idea as num_terms in layer_ctrl.v).
//
// Counting order, innermost to outermost:
//     k   : step inside one row of the window        (0 .. kernel-1)
//     ch  : which input row / channel                (0 .. num_channels-1)
//     pos : where the window sits (the "position")   (0 .. num_positions-1)
//     filt: which filter / neuron                    (0 .. num_filters-1)
// k and ch together are the "tap" (the step inside one answer):
//     tap = ch * kernel + k      (this equals layer_ctrl's term_idx)
//
// Addresses (relative to the start of the layer's own data and weights):
//     data_addr   = ch * row_len + pos + k
//     weight_addr = filt * (num_channels*kernel) + tap
//
// FC layers use the same module with num_positions = 1 and num_channels = 1.
// Then kernel is simply "how many inputs the neuron looks at"
// (992 for fc1, 32 for fc2) and data_addr = tap.
//
// No multipliers: instead of computing ch*row_len and filt*taps each time,
// the module keeps running base addresses and just adds to them.
//
// How it connects to layer_ctrl.v:
//     step        <- layer_ctrl enable   (a tap is being consumed this cycle)
//     next_output <- layer_ctrl done     (one answer has just been finished)
// step and next_output must never be high in the same cycle (they are not:
// enable is high in RUN, done is high in DONE).
//
// The address outputs show the tap being used THIS cycle and change at the
// clock edge. filter_idx / position_idx stay on the output just finished
// until the end of the done cycle, so whatever stores the result can read
// them while done is high (that is also where the bias is looked up).
//
// Requirements: kernel, num_channels, num_positions and num_filters must be
// at least 1.
//
module addr_gen #(
    parameter F_W  = 6,    // filter/neuron counter width  (holds up to 63;  needs 32)
    parameter P_W  = 9,    // position counter width       (holds up to 511; needs 256)
    parameter C_W  = 4,    // channel counter width        (holds up to 15;  needs 8)
    parameter K_W  = 10,   // kernel counter width         (holds up to 1023; needs 992)
    parameter DA_W = 11,   // data address width           (holds up to 2047; needs 1024)
    parameter WA_W = 16    // weight address width         (holds up to 65535; needs 31,744)
)(
    input  wire             clk,
    input  wire             rst_n,
    input  wire             init,           // synchronous: go back to the very first tap
    input  wire             step,
    input  wire             next_output,

    input  wire [F_W-1:0]   num_filters,
    input  wire [P_W-1:0]   num_positions,
    input  wire [C_W-1:0]   num_channels,
    input  wire [K_W-1:0]   kernel,
    input  wire [DA_W-1:0]  row_len,        // length of one input row (128 for conv2); unused when num_channels = 1

    output wire [DA_W-1:0]  data_addr,
    output wire [WA_W-1:0]  weight_addr,
    output wire [F_W-1:0]   filter_idx,
    output wire [P_W-1:0]   position_idx,
    output wire             last_output     // high while working on the final output of the layer
);

    reg [K_W-1:0]  k;
    reg [C_W-1:0]  ch;
    reg [P_W-1:0]  pos;
    reg [F_W-1:0]  filt;
    reg [DA_W-1:0] cbase;     // ch * row_len, kept as a running total
    reg [WA_W-1:0] waddr;     // the weight box to read now
    reg [WA_W-1:0] wbase;     // first weight box of the current filter

    wire k_last    = (k    == kernel        - 1'b1);
    wire ch_last   = (ch   == num_channels  - 1'b1);
    wire pos_last  = (pos  == num_positions - 1'b1);
    wire filt_last = (filt == num_filters   - 1'b1);
```

```
 81      wire tap_last  = k_last && ch_last;     // the final multiply of one answer
 82
 83      // pos and k are narrower than the address, so they are padded with zeros
 84      // explicitly (they are unsigned counters; no sign extension wanted)
 85      assign data_addr    = cbase + {{(DA_W-P_W){1'b0}}, pos} + {{(DA_W-K_W){1'b0}}, k};
 86      assign weight_addr  = waddr;
 87      assign filter_idx   = filt;
 88      assign position_idx = pos;
 89      assign last_output  = pos_last && filt_last;
 90
 91      always @(posedge clk or negedge rst_n) begin
 92          if (!rst_n) begin
 93              k <= 0; ch <= 0; pos <= 0; filt <= 0;
 94              cbase <= 0; waddr <= 0; wbase <= 0;
 95          end else if (init) begin
 96              k <= 0; ch <= 0; pos <= 0; filt <= 0;
 97              cbase <= 0; waddr <= 0; wbase <= 0;
 98          end else begin
 99
100              // ---- one tap consumed: move to the next tap ----
101              if (step) begin
102                  if (!tap_last) begin
103                      waddr <= waddr + 1'b1;                // next weight of this filter
104                      if (!k_last)
105                          k <= k + 1'b1;                    // next step along the window
106                      else begin
107                          k     <= 0;                       // window row finished, next channel
108                          ch    <= ch + 1'b1;
109                          cbase <= cbase + row_len;
110                      end
111                  end else begin
112                      k <= 0; ch <= 0; cbase <= 0;          // answer finished: back to tap 0
113                      if (!pos_last)
114                          waddr <= wbase;                   // same filter, next position: same weights again
115                      else if (!filt_last) begin
116                          waddr <= waddr + 1'b1;            // next filter's first weight
117                          wbase <= waddr + 1'b1;
118                      end else begin
119                          waddr <= 0;                       // whole layer finished: wrap
120                          wbase <= 0;
121                      end
122                  end
123              end
124
125              // ---- one answer finished: move to the next position / filter ----
126              if (next_output) begin
127                  if (!pos_last)
128                      pos <= pos + 1'b1;
129                  else begin
130                      pos  <= 0;
131                      filt <= filt_last ? {F_W{1'b0}} : filt + 1'b1;
132                  end
133              end
134          end
135      end
136
137  endmodule
```

B.5 mac_unit.v (31 lines)

```
 1  module mac_unit #(
 2      parameter DATA_WIDTH   = 16,   // one sample (e.g. Q4.12 or Q1.15)
 3      parameter WEIGHT_WIDTH = 16,   // one weight (e.g. Q2.14 or Q1.15)
 4      parameter ACC_WIDTH    = 48    // running total — wider to avoid overflow
 5  )(
 6      input  wire                             clk,
 7      input  wire                             rst_n,      // active-low reset
 8      input  wire                             clear_acc,  // start a fresh sum (new output channel)
 9      input  wire                             enable,     // do one multiply-add this cycle
10      input  wire signed [DATA_WIDTH-1:0]     data_in,
11      input  wire signed [WEIGHT_WIDTH-1:0]   weight_in,
12      output reg  signed [ACC_WIDTH-1:0]      acc_out
13  );
14
15      wire signed [DATA_WIDTH+WEIGHT_WIDTH-1:0] product;
16      assign product = data_in * weight_in;
```

```

    // sign-extend the 32-bit product up to the 48-bit accumulator width
    wire signed [ACC_WIDTH-1:0] product_ext =
        {{(ACC_WIDTH-(DATA_WIDTH+WEIGHT_WIDTH)){product[DATA_WIDTH+WEIGHT_WIDTH-1]}}, product};

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            acc_out <= 0;
        else if (clear_acc)
            acc_out <= enable ? product_ext : 0;   // start new sum
        else if (enable)
            acc_out <= acc_out + product_ext;       // keep accumulating
    end

endmodule
```

B.6 bias_rescale.v (63 lines)

```
// bias_rescale.v
// Closes the two gaps left open by mac_unit.v (see its report, Section 12):
// bias addition and inter-layer rescaling. Also applies ReLU, since it's a
// one-line check on the same value and not worth a separate module.
//
// Purely combinational: one shared instance serves every layer. Which layer
// is selected by what's fed into bias_shift / out_shift / apply_relu, the
// same "parameters as inputs, not per-layer copies" idea as layer_ctrl's
// num_terms.
//
//   acc_in      : the finished acc_out from mac_unit (Q-format = sum of the
//                 two operand formats, e.g. Q26 for conv1, Q29 elsewhere)
//   bias_in     : this output channel's bias, already in Q2.14 or Q1.15
//   bias_shift  : how far to shift bias_in left to match acc_in's format
//                 (12 for conv1, 14 for conv2/fc1/fc2)
//   out_shift   : how far to shift the sum right to land on Q2.14
//                 (12 for conv1, 15 for conv2/fc1/fc2)
//   apply_relu  : 1 for conv1/conv2/fc1, 0 for fc2 (raw class scores)
//   result      : final Q2.14 activation, saturated to 16-bit signed range
//   overflow    : 1 if saturation actually clamped something this cycle
//                 (diagnostic only - flags the activation-range question
//                 left open in the Phase 2.1 report, Section 7.3)
//
module bias_rescale #(
    parameter ACC_WIDTH = 48,
    parameter OUT_WIDTH = 16
)(
    input  wire signed [ACC_WIDTH-1:0] acc_in,
    input  wire signed [OUT_WIDTH-1:0] bias_in,
    input  wire [4:0]                  bias_shift,
    input  wire [4:0]                  out_shift,
    input  wire                        apply_relu,
    output wire signed [OUT_WIDTH-1:0] result,
    output wire                        overflow
);

    // Sign-extend the bias to the accumulator's width, then shift it up to
    // match acc_in's fractional-bit count (same "fractional bits add" rule
    // from Phase 2.1, run in reverse to align two different formats).
    wire signed [ACC_WIDTH-1:0] bias_ext     = {{(ACC_WIDTH-OUT_WIDTH){bias_in[OUT_WIDTH-1]}}, bias_in};
    wire signed [ACC_WIDTH-1:0] bias_shifted = bias_ext <<< bias_shift;

    wire signed [ACC_WIDTH-1:0] summed  = acc_in + bias_shifted;

    // Arithmetic right shift on a signed value floors toward negative
    // infinity, same as Python's integer >> - this must match, or hardware
    // and the golden model disagree on every negative activation.
    wire signed [ACC_WIDTH-1:0] rescaled = summed >>> out_shift;

    localparam signed [ACC_WIDTH-1:0] MAXV =  (1 <<< (OUT_WIDTH-1)) - 1;  //  32767 for OUT_WIDTH=16
    localparam signed [ACC_WIDTH-1:0] MINV = -(1 <<< (OUT_WIDTH-1));      // -32768

    wire ovf_hi = (rescaled > MAXV);
    wire ovf_lo = (rescaled < MINV);
    wire signed [ACC_WIDTH-1:0] clamped = ovf_hi ? MAXV : ovf_lo ? MINV : rescaled;

    wire signed [OUT_WIDTH-1:0] relu_applied =
        (apply_relu && clamped[ACC_WIDTH-1]) ? {OUT_WIDTH{1'b0}} : clamped[OUT_WIDTH-1:0];

    assign result   = relu_applied;
    assign overflow = ovf_hi | ovf_lo;

endmodule
```

B.7 max_pool.v (30 lines)

```
// max_pool.v
// Max-pooling (window=2), reduced to its actual hardware primitive: compare
// two neighboring activations, keep the larger one. One shared instance is
// reused sequentially across every pooling pair conv1/conv2 produce, the
// same "one small verified block, reused" approach as mac_unit.v and
// layer_ctrl.v, rather than building a wide parallel comparator tree.
//
// Purely combinational, like bias_rescale.v: a comparison needs no memory
// of previous cycles.
//
// In the real pipeline this only ever runs AFTER ReLU (see bias_rescale.v),
// so both inputs happen to be non-negative in practice. The module itself
// makes no such assumption and correctly compares any two signed 16-bit
// values, including negative ones, so it is not silently dependent on how
// it happens to be used today.
//
//   a, b    : the two neighboring activations being pooled
//   max_val : the larger of the two (b is returned on a tie)
//
module max_pool #(
    parameter DATA_WIDTH = 16
)(
    input  wire signed [DATA_WIDTH-1:0] a,
    input  wire signed [DATA_WIDTH-1:0] b,
    output wire signed [DATA_WIDTH-1:0] max_val
);

    assign max_val = (a > b) ? a : b;

endmodule
```

B.8 pool_pair.v (67 lines)

```
// pool_pair.v
// The "pooling holder": max-pool (window 2) for ONE output stream that arrives
// one number at a time, the way our single shared MAC unit produces it.
//
// Why it exists
//   Max-pool keeps the bigger of two NEIGHBOURING conv outputs (positions 2p and
//   2p+1 of the same filter). The MAC unit finishes them one after the other,
//   so when the first one appears its partner does not exist yet. This module
//   parks the first number in one register and, when the second arrives, hands
//   both to max_pool and passes the winner on. The unpooled outputs are never
//   stored anywhere.
//
// How it works (every number is one finished, post-ReLU output)
//   in_valid = 1, pos_odd = 0 : first of a pair  -> copy in_val into 'held'
//                                                   out_valid = 0 (nothing to write yet)
//   in_valid = 1, pos_odd = 1 : second of a pair -> out_val = larger of 'held' and in_val
//                                                   out_valid = 1 (write out_val now)
//   in_valid = 0              : nothing happens; 'held' keeps its value for as
//                               long as it takes (idle clocks between outputs are fine)
//
//   pos_odd is simply the lowest bit of the output's position number
//   (0 = even = first of a pair, 1 = odd = second). Because every number says by
//   itself whether it is first or second, there is no toggle flag inside this
//   module that could ever fall out of step.
//
// Timing
//   Inputs are used in the same clock they are presented, nothing is delayed:
//   out_valid / out_val are combinational (like max_pool and bias_rescale), so
//   the pooled value is available in the SAME clock as the second number and the
//   write into the buffer can happen at that clock edge. The only state is 'held'.
//
// Rules for the user of this module
//   - in_valid must be 1 for exactly one clock per finished output, with in_val
//     and pos_odd belonging to that same output.
//   - A pair is always (even position, then the next odd position) of the same
//     filter, so the number of positions per filter must be even
//     (conv1: 256, conv2: 124 - both are).
//   - There is no reset: 'held' is unknown until the first pair starts, and it is
//     never used before that (out_valid is 0 until a second number arrives).
//   - The pooled position is p = position / 2, i.e. position[MSB:1].
```

```
41  //
42  module pool_pair #(
43      parameter DATA_WIDTH = 16
44  )(
45      input  wire                         clk,
46      input  wire                         in_valid,   // 1 = in_val is a finished output this clock
47      input  wire                         pos_odd,    // lowest bit of that output's position
48      input  wire signed [DATA_WIDTH-1:0] in_val,     // the finished (post-ReLU) output
49      output wire                         out_valid,  // 1 = out_val is a finished pooled value
50      output wire signed [DATA_WIDTH-1:0] out_val     // larger of the pair (meaningful only when out_valid = 1)
51  );
52
53      reg signed [DATA_WIDTH-1:0] held;               // the parked first number of a pair
54
55      always @(posedge clk)
56          if (in_valid && !pos_odd)
57              held <= in_val;
58
59      max_pool #(.DATA_WIDTH(DATA_WIDTH)) compare (
60          .a      (held),
61          .b      (in_val),
62          .max_val(out_val)
63      );
64
65      assign out_valid = in_valid && pos_odd;
66
67  endmodule
```

B.9 weight_rom.v (41 lines)

```
 1  // weight_rom.v
 2  // Read-only memory holding ALL 32,488 weights of the network (conv1, conv2,
 3  // fc1, fc2 stored one after another), loaded from weights.hex.
 4  //
 5  // It is meant to become FPGA block RAM (BRAM). Block RAM reads are
 6  // synchronous: you present an address, and the weight shows up on `data`
 7  // ONE CLOCK EDGE LATER, then stays there until the next edge.
 8  //
 9  //     cycle n   : addr = A
10  //     cycle n+1 : data = weight[A]        (addr may already be showing a new value)
11  //
12  // That one-cycle delay is why layer_ctrl's clear_acc / enable / done, and the
13  // result's write address, will have to be delayed by one cycle when this ROM
14  // is wired in (done at the conv1 wiring step, not here).
15  //
16  // Layer start addresses inside this ROM (the same numbers export_rom_files.py prints):
17  //     conv1 = 0     conv2 = 40     fc1 = 680     fc2 = 32424
18  // addr_gen's weight_addr is relative to its own layer, so whoever wires this
19  // ROM adds the layer's start address first.
20  //
21  // Addresses 32,488 and above do not exist. addr_gen never produces them;
22  // the real hardware returns unspecified data for them (simulation shows x).
23  //
24  module weight_rom #(
25      parameter DEPTH     = 32488,                // number of weights
26      parameter AW        = $clog2(DEPTH),        // address width: 15 bits (covers 0..32,767)
27      parameter INIT_FILE = "weights.hex"         // one 4-digit hex weight per line
28  )(
29      input  wire                clk,
30      input  wire [AW-1:0]       addr,
31      output reg  signed [15:0]  data
32  );
33
34      (* rom_style = "block" *) reg signed [15:0] mem [0:DEPTH-1];
35
36      initial $readmemh(INIT_FILE, mem);
37
38      always @(posedge clk)
39          data <= mem[addr];
40
41  endmodule
```

B.10 bias_rom.v (35 lines)

```
// bias_rom.v
// Read-only memory holding the 58 biases (conv1 8, conv2 16, fc1 32, fc2 2,
// stored one after another), loaded from biases.hex.
//
// Unlike weight_rom, this read is INSTANT (combinational): change addr and
// data follows in the same cycle, no clock involved. 58 numbers is small
// enough that the FPGA builds it from ordinary logic (LUTs) instead of
// block RAM, so there is no one-cycle delay to compensate for.
//
// Where each layer's biases start (the same numbers export_rom_files.py prints):
//     conv1 = 0     conv2 = 8     fc1 = 24     fc2 = 56
// The address for a given output is: layer start + filter_idx.
// addr_gen holds filter_idx steady while layer_ctrl's done is high, which is
// exactly when bias_rescale needs the bias.
//
// Addresses 58..63 do not exist (simulation shows x for them).
//
module bias_rom #(
    parameter DEPTH     = 58,
    parameter AW        = $clog2(DEPTH),         // 6 bits
    parameter INIT_FILE = "biases.hex"           // one 4-digit hex bias per line
)(
    input  wire [AW-1:0]       addr,
    output wire signed [15:0]  data
);

    // No memory-style hint on purpose: block RAM cannot do an instant read, so
    // the tools build this table from LUTs by themselves (16 LUT6, one per output bit).
    reg signed [15:0] mem [0:DEPTH-1];

    initial $readmemh(INIT_FILE, mem);

    assign data = mem[addr];

endmodule
```

B.11 data_buffer.v (55 lines)

```
// data_buffer.v
// A small memory for DATA (as opposed to weight_rom / bias_rom, which hold the
// fixed trained numbers). The same module is used four times, with a different
// DEPTH each time:
//
//     input buffer      DEPTH = 260    the heartbeat's samples (Q4.12)
//     activation A      DEPTH = 1024   conv1 output after ReLU + pooling  (8 x 128)
//     activation B      DEPTH = 992    conv2 output after ReLU + pooling  (16 x 62), already flattened
//     activation C      DEPTH = 32     fc1 output
//
// One write port and one read port, both on the same clock.
//
//   Write : when we = 1, wdata is stored at waddr on the next clock edge.
//   Read  : the address is given on raddr; the value appears on rdata ONE CLOCK
//           EDGE LATER (registered read), and then stays until the next edge.
//           This is the same one-cycle delay as weight_rom, so a sample read
//           from here and a weight read from weight_rom arrive together.
//
//   Read and write of the SAME address in the SAME clock: the read returns the
//   OLD value (read-first); the new value shows up on the following read.
//   The design never does this on purpose (a layer reads one buffer while it
//   writes a different one), but the behaviour is fixed so it is never a surprise.
//
//   There is no reset and no initial content: a location that was never written
//   holds nothing meaningful (simulation shows x; real hardware may show 0 or old
//   data). Never read a location before the layer before it has written it.
//
//   Addresses must stay inside 0 .. DEPTH-1. Outside that range simulation shows x
//   and ignores the write, but real hardware may behave differently (the 992-word
//   buffer, for instance, sits in a 1024-word block RAM), so never use such addresses.
//
// The tools decide whether each instance becomes block RAM or LUT RAM. Because
// the read is registered in the code, both behave identically clock for clock.
//
module data_buffer #(
    parameter DEPTH = 1024,
    parameter AW    = $clog2(DEPTH)       // address width (DEPTH = 1 is not supported)
)(
    input  wire               clk,
    input  wire               we,
```

```
41      input  wire [AW-1:0]       waddr,
42      input  wire signed [15:0]  wdata,
43      input  wire [AW-1:0]       raddr,
44      output reg  signed [15:0]  rdata
45  );
46
47      reg signed [15:0] mem [0:DEPTH-1];
48
49      always @(posedge clk) begin
50          if (we)
51              mem[waddr] <= wdata;
52          rdata <= mem[raddr];             // sees the old value when waddr == raddr in this cycle
53      end
54
55  endmodule
```

B.12 decision.v (20 lines)

```
 1  // decision.v
 2  // The final decision of the network: which of the two class scores is higher.
 3  //
 4  //     score0 = score of class 0 (Normal)       score1 = score of class 1 (Abnormal)
 5  //     abnormal = 1   when score1 is HIGHER than score0
 6  //     abnormal = 0   when score0 is higher, and also when they are EQUAL (a tie means Normal)
 7  //
 8  // The scores are signed 16-bit numbers (Q6.10) and can be negative, so the comparison is a
 9  // SIGNED one. Purely combinational: no clock, the answer follows the inputs immediately.
10  // (Same rule as the Python model: Abnormal only if score1 > score0.)
11  //
12  module decision (
13      input  wire signed [15:0] score0,    // class 0, Normal
14      input  wire signed [15:0] score1,    // class 1, Abnormal
15      output wire               abnormal   // 1 = Abnormal, 0 = Normal
16  );
17
18      assign abnormal = (score1 > score0);
19
20  endmodule
```